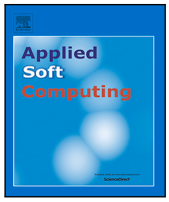
Applied Soft Computing Journal 167 (2024) 112239



Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc





Decomposition based cross-parallel multiobjective genetic programming for symbolic regression

Lei Fan [a,*], Zhaobing Su [b,a], Xiyang Liu [a], Yuping Wang [a]

[a] School of Computer Science and Technology, Xidian University, Xi'an 710071, China
[b] Northwest Institute of Mechanical & Electrical Engineering, Xianyang 712099, China

ARTICLE INFO

Keywords:
Symbolic regression
Problem decomposition
Global regression
Local regression
Cross-parallel multiobjective genetic programming

ABSTRACT

Genetic Programming (GP) based Symbolic Regression (SR) algorithms suffer from the ineluctable effects over model bloat, blind search and diversity loss when determining explicit symbolic models to best depict the concealed laws in historical data, which often make them time-consuming and unstable. Most efforts often dealt with one of these effects, such that the algorithms still suffer from other effects. To deal with these effects, we propose a cross-parallel SR algorithm framework based on problem decomposition and multiobjective GP in this paper. The decomposition method is proposed to distill simple subproblems named global and local regression, which can be fast solved to produce various high quality models. In the proposed framework, by expressing the SR problem as the multiobjective optimization model, a number of subproblems are automatically distilled and solved in parallel to reduce model bloat and accelerate the algorithm. Traditional regression methods are employed to produce high quality models to seed the evolving populations for each subtask to maintain population diversity and improve search efficiency. Elite models obtained by each subtasks will be collected and randomly sent to other subtasks to improve the model generalization. Ablation and comparison experiments are conducted to evaluate the performance of the proposed algorithms. The ablation results show that the developed algorithm framework plays a positive role in reducing above ineluctable effects, and can fast determine concise symbolic models for the benchmarks. Comparisons by SRBench demonstrate the effectiveness of the developed algorithm on wide range problems.

1. Introduction

Genetic Programming (GP) based Symbolic Regression (SR) algorithms aim to distill explicit symbolic models to gain interpretable knowledge from historical data [1–11], which have attracted ever-increasing attention and found wide applications over distilling natural laws, simplifying expressions, data mining, and model abstracting [12–15]. A number of effective algorithms have been developed in the last decade [1–3,7,9–11,16–23]. Different from traditional regression methods (e.g., linear regression, polynomial regression, nonlinear regression), GP evolves a set of symbolic models that usually represented as mathematical expressions by using specialized genetic operators to automatically determine both forms and coefficients of the target models [8,24,25], which is known to be NP-hard [26,27] and has following characters: (1) there are usually many equivalent models with various forms that can best fit the data or depict the underlying regularity; (2) the search spaces are composed of numerous discrete models; (3) mutual effects exist between model form and coefficients. Such characters often make difficulties for solution algorithms to find reasonable models and lead to ineluctable effects for GP based SR algorithms. However, most efforts devoted to improving algorithm performance often focused on dealing with one of these effects, such that the other effects still influence the algorithm. Moreover, rapidly increasing volume and complexity of the data in almost every practical field bring more and more difficulties in determining reasonable models and, appeal to effective SR algorithms. Hence, this paper studies the GP based SR algorithm for large-scale data with the purpose of simultaneously dealing with several effects.

The ineluctable effects occurred when using GP to solve SR problems mainly lie in model bloat (an excess of the model size growth without a corresponding improvement in fitness) [1,2,28–33], blind search (refers to lack of search directions for improving the evolving model) [34,35], diversity loss [14] (there tends to be no significant difference in fitness between individuals of the population) and overfitting (occurred when the obtained models are with high training (fitting) accuracy but low prediction accuracy) [36–38], which significantly affect the performance and limit the applications of the solution algorithms.

* Corresponding author.
E-mail address: lfan@mail.xidian.edu.cn (L. Fan).

https://doi.org/10.1016/j.asoc.2024.112239
Received 27 February 2024; Received in revised form 26 August 2024; Accepted 4 September 2024
Available online 17 September 2024
1568-4946/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

The model bloat effect may lead to excessive processor consumption, slow model evaluation and unreadable models [28]. The effects of blind search and diversity loss often lead to time consuming or slow convergency of symbolic regression algorithms.

Efforts [28–42] have been dedicated to improving the performance of SR algorithms through dealing with these undesired effects or accelerating the algorithm running speed, which arose many effective strategies and will be briefly summarized in Section 2. Some specific strategies and operators have been developed to deal with the effects [8], such as coevolution strategies [30,32,33], dynamic limits [31] and operator equalization [29] for model bloat, syntax analysis [35] and local search [39,43] for blind search and, the first order derivative of GP trees [42] and nonlinearity control [38] for overfitting. However, these operators or strategies are developed for only one of these effects, such that some other contingent effects may occur. For example, model bloat effect occurred in Geometric Semantic Genetic Programming (GSGP) [10,11] while using the geometric semantic to improve the search efficiency, which is studied continually [1,2,7,9,16]. Too much emphasis on fitting precision may lead to overfitting [8,14,36], which prompts the SR methods to be able to discern the overfitting effect and distill the true models. To this end, the decomposition techniques [6, 44–46] were integrated into GP-based SR algorithms for the purposes of comprehensively distilling and assessing symbolic models. However, the decomposition based SR methods suffer from heavy computation in evaluating models for large size training data. Aiming to reduce such computation pressure, the sub-datasets randomly sampled from the training data were used to evaluate the evolving symbolic models of GP in [13,47–51], which were even used to evaluate sampled individuals (symbolic models) in [52,53]. The strategy of using sub-datasets to train symbolic models was further studied in [54–56] and, shown to be effective on preventing the overfitting effect. Nevertheless, using the specific random sub-dataset to train or evaluate the evolving symbolic models has significant impacts and biases on fast seeking out comprehensive symbolic models to best fit the original data. Moreover, to improve the time efficiency of GP-based SR algorithms, acceleration techniques have been developed, such as accelerating model evaluation [13,48,50,51,57–60] and algorithm parallelism [14,44,61]. These techniques have little regard for above effects. While parallelizing solution algorithms, each subtask usually solves the original SR problem. To our knowledge, little attention had been paid to decomposing the SR problem into simple subproblems and dealing with as many effects as possible. Different from previous efforts, we focus on decomposing the SR problem and developing the high-efficiency SR algorithm in this paper to deal with as many effects as possible.

Inspired by the successful experiences of manual data analysis, we proposed a cross-parallel SR algorithm framework based on problem decomposition and multiobjective genetic programming (MoGP) to deal with multiple effects simultaneously in this paper. The main contribution of this paper is summarized as follows:

(1) We surveyed the efforts devoted to improving the performance of SR algorithms briefly in terms of effect control and algorithm acceleration for the first time, in which we summarized four kinds of effects including model bloat, blind search, overfitting and diversity loss and two categories of algorithm accelerating strategies including accelerating model evaluation and algorithm parallelism.
(2) A decomposition method is proposed to decompose the SR problem into simple subproblems, named global regression (GR) and local regression (LR), which can be fast solved to produce various high quality models and improve or train symbolic models mutually.
(3) A cross-parallel SR algorithm framework is proposed based on the decomposition method and MoGP. A set of subproblems including different GR and LR subproblems are automatically established by sampling different subsets from original dataset. Each subproblem is expressed as a multiobjective optimization model and solved by the MoGP. During the evolving process, the elite models of each subproblem will be trained and improved by different subtasks.
(4) To improve the performance of proposed algorithm framework, multithreading and traditional regression methods are employed. Via multithreading, each subproblem is solved in an independent thread. The traditional regression methods are employed to produce high-quality models for subproblems, which will be used to seed the evolving population. Elite models of subproblems will be collected and randomly selected to seed the population of each subproblem.
(5) Ablation experiments on three groups of benchmarks and comparisons based on SRBench [25] are conducted to evaluate the performance of the proposed method.

The rest of the paper is organized as follows: Section 2 briefly summarizes the related work on GP-based SR algorithms in terms of effect control and algorithm acceleration. Section 3 states the multiobjective optimization model for symbolic regression. The motivations and task decomposition strategy are presented in Section 4, in which the global and local regression based on sampling methods are proposed. In Section 5, the proposed algorithm framework for SR is developed. Experiments and comparisons are conducted in Section 6. At last, Section 7 draws the conclusion of this paper.

2. Related work

Many approaches [1,2,5,12,28–34,62–68] for SR problems have been developed and found their wide applications, such as Boolean function construction [69], physical science [12] and feature selection [70,71]. A. Agapitos et al. surveyed the studies on GP-based SR algorithms in [8]. In the last decade, many effective SR algorithms were proposed, such as Geometric Semantic Genetic Programming (GSGP) [1,2,7,9–11,16], Generalized Symbolic Regression (GSR) [3], piecewise symbolic regression tree (PS-Tree) [17], GP version of the gene-pool optimal mixing evolutionary algorithm (GP-GOMEA) [18], Model Learning with Personalized Interpretability Estimation (ML-PIE) [19], Koza-style symbolic regression in Python (gplearn) [20], interaction-transformation evolutionary algorithm (ITEA) [21], SR with non-linear least squares [72], the genetic programming framework in C++for SR (Operon) [73], semantic back-propagation genetic programming (SBP-GP) [22], multiple regression genetic programming (MRGP) [23] and so on. Their excellent performance on recognition and prediction of important variables or features implies their potential powerful practical applications [70,74–77]. The efforts devoted to improving the performance of SR algorithms will be briefly surveyed in terms of effect control and algorithm acceleration in this section.

2.1. Effect control

Various techniques and strategies have been proposed to reduce the effects over model bloat, blind search, losing diversity and overfitting as mentioned in Section 1 aiming at improving the performance of SR solution algorithms. A majority of these efforts focused on the model bloat. Effective techniques or strategies are still needed to be developed to deal with the other effects.

Model bloat Model bloat may cause problems over memory overflow, excessive model evaluation, slowing down the rate of model evolution, unreadable models and so on, which is a long active investigation topic in GP based SR algorithms [1,2,16,28,29,31,61,78–87]. The efforts devoted to this effect of a decade ago had been surveyed in [29,31,79]. In this period, this effect was discussed theoretically in [78,83,85–87] and handled in [79–81,84,88]. To control model bloat, many existing methods usually set a threshold of model size to restrict the size of candidate models [31], or penalize large models [79], such as parsimony pressure [84], size control [81], Tarpeian bloat control (refers to the bloat

reducing method based on the schema theory [89], which is essentially an operator for selecting above-average sized models) [80,90,91], double and proportional tournament (refers to proportionally selecting the smallest model from the set of models with best fitness that picked from each sampled subset) [28,79], operator equalization (refers to the bloat-control technique that dynamically changes the size distribution for a population such that more individuals can be admitted to produce fitter models) [29,88], and so on. Such size constraints are usually relatively unstrict [88,92] for the sake of temporarily keeping diverse genetypes. However, while dealing with model bloat, arbitrarily penalize or eliminate the oversized models without considering their other attributes may omit useful or informative genetypes and slow down the model evolving process. To handle such shortcomings, genetic programming algorithms assembled with multiobjective optimization techniques are commonly used to deal with model bloat by expressing SR problems as multiobjective optimization problems (MOPs) with minimizing model size as one objective [14,19,82]. In this way, the SR solution algorithms should search for concise Pareto optimal models with other best attributes. In [14], the SR algorithm was adopted to produce a small set of accuracy–parsimony Pareto expressions, in which the optimal solutions vary over their complexity (size) and the predictive precision. The SR problem studied in [38] was represented as a multiobjective optimization problem with three objectives: accuracy, the non-linearity order and expressional complexity (size). Although there may be some nondominated models during solving MOP based SR algorithms, keeping diversity and speeding convergence still trouble the algorithms.

Besides above common used bloat control techniques, some efforts used the model evaluation time to indicate the complexity and manage bloat according to the correlation between the model size and relevant evaluation time [93,94]. However, the evaluation time varies with the operators, which may cause longer evaluation time of small sized models than larger ones. Hence, this technique may omit some concise models when evolving models. For tree-based GP algorithms for SR problems, some genetic operators were proposed to handle model bloat, such as hoist mutation [95], shrink mutation [96] and permutation mutation [35]. The similar idea of these operators is to use small sized subtree to replace the subtree random selected from the parent model. The model bloat occurred in GSGP [10,11,16] due to the fact that the genetic operators called Geometric Semantic Operators (GSOs) consistently generate offspring larger than their parents [1,2,7,9]. Recently, L. Vanneschi [1] proposed a varied GSGP called Semantic Learning algorithm with Inflate and deflate Mutations (SLIM_GSGP), in which the deflate mutation was used to delete several previously added terms or genotypes aiming at controlling model bloat. Although the experiment results in [1] show that SLIM_GSGP performed better than GSGP in bloat control, the model sizes are still quite large and, the results on data fitting abilities of the obtained models show that the algorithm performed worse than GSGP and standard GP. Hence, further efforts should be devoted to improve the performance of SLIM_GSGP on finding high quality models to best fit the data.

Blind search This effect is derived from [11]. According to the characters of SR problems, this effect is essentially due to that solution models in the search space are discrete and it is difficult to capture the relationships among them syntactically. The search operators of traditional GP-based SR algorithms usually focused on syntactic representations of solution models, regardless of their other attributes, such as semantics [11]. As a typical blind syntactic search operator, the subtree swap crossover recombines the models represented as parse trees and only guarantees producing syntactically well-formed expressions. Endeavors to this effect yielded some techniques to improve the search efficiency, e.g., multiobjective optimization [19,41], the stepwise adaptive weights method [34], syntax analysis [35] and semantic-awareness methods [1,2,97–105]. In [41], a multiobjective evolutionary algorithm was developed for SR problems by using the island-population model and a fitness-prediction model to reduce computational cost and improve the local search gradient. Study on this topic is still limited [8].

Among these efforts, semantically methods have been shown to be effective in improving the search efficiency of GP-based SR algorithms [106–108], which arises many genetic operators [16,106,108,109] and algorithms [1,2,11,97]. As one of the popular GP framework, GSGP uses geometric semantic operators and the geometry of model space to search for solution models by syntactic manipulation to minimize the fitness value according to the distance between the solution and the target output vector. The ongoing efforts devote to improve the performance of GSGP via semantic crossover or mutation operators. The semantic-aware crossover operators proposed by Uy et al. [97,108] swap the subtrees with similar semantics in the two parents to produce offspring, such that high locality can be maintained for a better generalization of GP. Furthermore, the bounded geometric mutation [110], geometric semantic mutation [109] and geometric semantic crossover with angle-aware mating [106] were proposed. As investigated by Gonçalves et al. [109], the geometric mutation performed better than the geometric crossover in terms of improving the generalization ability of GSGP. Although all studies showed that the semantically aware methods above produced symbolic models with the prediction ability on unseen data superior to traditional methods [40], the bloat effect still troubles GSGPs [1]. Geometric search operators were proposed in [16] to fulfill semantic requirements, in which important nodes of expression trees were recognized via the partial derivative value and set priority set priority for further modification in crossover. Although these operators were showed to be effective in improving the performance of GP-based SR algorithms, a lot of model evaluation will be needed when producing offspring. The two drawbacks [11] over heavy computation and no insights on the relationship between syntactic and semantic searches are still needed to be considered and handled while improving algorithm performance. Moreover, effective evolving directions for improving models is another significant issue in improving GP-based SR algorithm performance.

Overfitting Overfitting is often caused by noisy or low-quality data, which is an active topic in machine learning and data mining [2,8,37,49,61,111–115]. The overfitting effect in SR is usually referred to the loss of generalization of models [116]. A measurement of overfitting evaluation for GP was proposed in [37]. Literature reviews of research efforts devoted to overfitting problem in symbolic regression can be referred to [2,49,111,115]. There are two opposite perspectives on whether it has relationship between model bloat and overfitting. One perspective is that the overfitting effect has correlation with bloat due to the perception that bloated models seem to be more likely to overfit data than those with a smaller size [37,49,58,117]. Following this perspective, some researchers proposed complexity measures for GP trees to improve generalization ability and avoid overfitting, such as the first order derivative of GP trees [117], variance [118], Tikhonov regularization [119], curvature estimation [37], nonlinearity estimation [38] and variance functional [120]. Whereas, the other perspective deems that model bloat and overfitting are in most part two independent phenomena due to researchers found that overfitting occurred in bloat-free environments [121,122]. The most recent Ref. [2] also supports this perspective and, reports that the popular GSGP (suffers from bloat effect) surpasses standard GP in limiting overfitting [11,123,124]. To avoid overfitting, França et al. argued that "whenever possible, overfitting is avoided by a careful examination of the nature of the data and using prior knowledge about the studied phenomena to select only the relevant features and make a conscious choice about the parametric model." [111]. Hence, an efficient technique for avoiding overfitting should take various issues into consideration comprehensively.

Some techniques utilize the subsets of data instances or populations to avoid overfitting. Previous works on utilizing subsets of training data have shown the effectiveness both in reducing the runtime and overfitting [58,59]. C. Gathercole [47] showed that dynamic subset selection can speed up genetic programming on the large, messy Thy-

roid problems. This technique was further used in [125] to reduce overfitting in software quality classification. These two works utilized the subsets with different sampling ratios, which prompted further effort on showing the abilities of smaller percentages of total training data on reducing overfitting and improving generalization via various problems [115,126,127]. Such techniques were additionally shown in [115] to be able to produce models with smaller size. The techniques called backwarding [128] and validation start [127] were proposed to prevent overfitting by utilizing the subset of population called validation set to select the best individuals. In [58], the authors handled overfitting based on the combination of validation set and random subset of training data. However, previous works did not pay attention to the special subsets of the training data that may be useful for solution algorithms to evolve exact and applicable models.

Diversity loss This effect is derived from [129]. Diversity is a key factor in the performance of GP-based SR algorithms and other evolutionary algorithms, and has been deeply studied for a long time [5,88,129–131]. The diversity measurement for GP is argued to be able to take into account both aspects of genotype and phenotype, which is a difficult task due to the complex relationship between these two aspects [112,132,133]. Since the blind search effect slows down GP-based SR algorithms in producing better offspring from the parents and the negative effects of genotype operations (particularly crossover) on individuals often lead to diversity loss, the outstanding model suddenly emerged may eliminate many individuals with useful genetypes or its own genetypes may prevent the genetic operators (the interplay between crossover and selection) from producing better individuals [88]. Hence, preserving diverse individuals can benefit the model evolving process in GP [112,134]. Previous studies dedicated to GP diversity analysis, diversity measurement and preservation are most in correlation with the genetic operators [5,37,98,112,129,131,133,135]. Many diversity criteria are based on distance or entropy metrics, which have strong correlations with individual fitness in GP [112,131]. In semantic genetic programming, semantic diversity is another important issue and has gained constant concerns on semantic diversity measurements and preservation [5,98,100,103,112,136–138]. Diversity preservation strategies are often relevant to genetic operators and designed in combination with semantic diversity metrics. It has been shown that semantically diverse initial populations could benefit to improve the GP performance [100,139]. D. Jackson defined semantic diversity based on the fitness distribution [98] and tree behavior at runtime [137] respectively. Q. Chen et al. [112] proposed an entropy-based semantic diversity measure based on the transformed semantics of models in GP, which was further used to design the fitness function and genetic operator to preserve diversity. Q. Chen also reviewed the related work on measuring and maintaining semantic diversity for GP [112]. Most previous works on diversity for GP focused on how to preserve diversity via designing selection strategies or genetic operators to produce diverse offspring, such as the random Boolean networks [140], hereditary repulsion (a convergence manipulation protocol) [141], angle-driven selection [40] and so on. The common goal is to make the best of preserved diverse genetypes. But few attention has been paid to increase the population diversity by introducing high-quality diverse models. To this end, we use traditional regression methods on different sub-datasets to fast produce diverse high-quality models to seed the evolving populations when needed.

2.2. Algorithm acceleration

Except the contributions to above effects for improving the performance of GP-based SR algorithms, some efforts devoted to improving the time efficiency via speeding up the algorithm execution, which are classified into two categories in this paper: accelerating model evaluation and algorithm parallelism.

Accelerating model evaluation Using the training data to assess the evolving models in the evolution process of GP is essentially a data and computing intensive task, especially for large-scale data. Efforts have been devoted to developing techniques or strategies to accelerate model evaluation aiming at reducing the computing pressure caused by large volume of the training data for GP-based SR algorithms, which are classified into two types in this paper. One is to use sampling methods (e.g., random sampling, dynamic subset selection) to select a subset of training data to evaluate individuals' fitness in each generation [13,47,48,50,51,57–61,142,143]. Although these works showed the effectiveness of using subset of training data to assess evolving models in speeding up GP-based SR algorithms, the evolving models cannot be assessed comprehensively via a subset of training data, which may lead to incorrect assessments on high-quality models. Hence, it needs to study how to select reasonable subsets for correct or unbiased assessments. The other is to use fast processors, such as graphics processing units (GPUs), to accelerate the task of model assessment based on hardware acceleration technologies. This means had been shown to be able to significantly reduce the run time of GP-based SR algorithms [57,60,144]. However, the solution models obtained by the solution algorithms still suffer from above effects.

Algorithm parallelism Algorithm parallelism is another effective strategy for accelerating GP-based SR algorithms by using multi-core processors and multithreading techniques, which often follows the parallelism of evolutionary algorithms (EAs) [14,142,143,145–148]. Some efforts [57,60,61,142,143] dedicated to this scenario only focused on the parallelization of model evaluation, such that these GP-based SR algorithms are in essence serial. The so called island model [149] is commonly employed in parallelizing EAs, which divides the evolving populations into several distinct islands, and each island evolves a subpopulation in a separate parallel thread [14,146]. The parallelism of the multi-population based GP framework with adaptively weighted primitives proposed in [150] is essentially the same as island model. In these developed GPs, each thread solves the original SR problem by the GP with smaller population, which may lead to slow convergence or low accuracy. In [45], the SR problem was expressed as the disjunctive programming problem, which will be approximated by the underestimation convex problem and solved by the generalized benders decomposition (GBD) based global optimization approach [46]. The GP-based SR algorithm proposed in [44] is mainly composed of two steps: k parallel subtasks are firstly performed to determine k intermediate models to fit the training data, which will then collected to seed the population of GP to determine the final symbolic model. This method actually solves the original SR problem $k + 1$ times. Although these proposed algorithms solve the SR problem in parallel, each subtask solves the same original problem and does not pay attention to fast producing high-quality models, which does not improve the algorithm performance actually. To parallelize GP-based SR algorithms, we propose a method to decompose SR problems into simple subproblems which can be fast solved in parallel. Meanwhile, different high-quality models can be obtained such that the model diversity can be improved. A cross-parallel GP is proposed to improve the obtained models by different subtasks.

2.3. SRBench

The SRBench [25] is a SR benchmark repository recently developed by integrating the benchmark datasets and contemporary methods, which can facilitate researchers and data engineers in developing, assessing or selecting SR methods. In [25], 21 contemporary methods for SR problems, including 14 SR algorithms and 7 machine learning algorithms, were assessed by using SRBench. The SR algorithms include age-fitness Pareto optimization (AFP) [146], AFP with co-evolved fitness estimates (AFP_FE) [14], physics-inspired method (AIFeynman) [151], Bayesian symbolic regression(BSR) [152], deep

symbolic regression (DSR) [153], ϵ-lexicase selection (EPLEX) [154], feature engineering automation tool (FEAT) [155], fast function extraction(FFX) [156], GP version of the gene-pool optimal mixing evolutionary algorithm (GP-GOMEA) [18], Koza-style symbolic regression in Python (gplearn), interaction-transformation evolutionary algorithm (ITEA) [21], multiple regression genetic programming (MRGP) [23], SR with non-linear least squares (Operon) [72], and semantic backpropagation genetic programming(SBP-GP) [22]. The machine learning algorithms include Scikit-Learn[1] including two popular gradient-boosting algorithms (XGBoost [157] and LightGBM [158]), Random Forest [159], AdaBoost [160], Kernel ridge regression (KernelRidge), Linear regression (Linear) and Multi-layer Perceptron (MLP). The results demonstrated that GP-based SR methods outperform new SR methods based in other fields of optimization. In this paper, SRBench is employed to conduct comparison experiments to evaluate the developed algorithm framework.

3. Multiobjective optimization model for symbolic regression

The historical data to be analyzed are often recorded as a set of tuples, and each tuple contains the value of independent variables (features) and the relevant value of dependent variables. Let $D = \{(\mathbf{x}_i, y_i)|i = 1, 2, \ldots, |D|\}$ be the set of all data instances, where $\mathbf{x}$ denotes the vector of n independent variables, y the recorded value relevant to $\mathbf{x}$ and $|D|$ the total number of instances. Distilling the concealed law in D can help make further prediction, which is often depicted by the symbolic model $\tilde{y} = f_D(\mathbf{x})$, where $f_D : \mathbb{R}^n \to \mathbb{R}$, and $\tilde{y}$ the value of f_D at $\mathbf{x}$. To this end, researchers usually determine optimal symbolic models, denoted as f_D^{opt}, to best fit the data in D, for which one has to determine both the form and coefficients. Such optimal symbolic model f_D^{opt} is commonly defined to be with a minimal deviation between the recorded value y in D and the fitting value $\tilde{y}$. This task can be naturally formulated as the optimization problem SR_{Prob} in Eq. (1), where $\|f_D(\mathbf{x}) - y\|$ denotes the integrated difference between $f_D(\mathbf{x})$ and y for all data instances in D, and $\mathbb{F}$ the symbolic model space.

$$(\text{SR}_{\text{Prob}}) \quad f_D^{\text{opt}} = \arg\min_{f_D \in \mathbb{F}} \|f_D(\mathbf{x}) - y\|_{(\mathbf{x},y)\in D} \tag{1}$$

Representing the symbolic regression problem as a multiobjective optimization model can help solution algorithms deal with the diversity loss effect [41]. In this paper, aiming at obtaining concise symbolic models that can best depict the true laws in the data, the symbolic regression problem is represented as a bi-objective optimization problem to minimize both training error and model size for the training data.

The measurements to assess model size for SR, such as syntactic complexity [161], semantic complexity [38,151,161] and meta-learning based complexity estimation [162], have been proposed and discussed [163]. A straightforward but effective measurement [25] is defined to be the number of symbolic elements in the model, such as mathematical operators, features and constants. In this paper, the evolving models are represented by binary trees, for which the model size is defined to be the number of nodes in binary trees, and calculated by Eq. (2). The training error is adopted with the common used Root Mean Squared Error (RMSE), and calculated by Eq. (3).

$$SIZE = node(f_D) \tag{2}$$

$$RMSE = err(f_D, D) = \sqrt{\frac{1}{|D|}\sum_{i=1}^{|D|}(f_D(\mathbf{x}_i) - y_i)^2} \tag{3}$$

The general form of the bi-objective optimization model for symbolic regression can be formulated as SR_{MOP} in Eq. (4). The models with relevant size within $[\min_{size}, \max_{size}]$ will be selected during evolving the models in this paper. According to Pareto theory of multiobjective optimization, there will be a number of nondominated models for SR_{MoP}, which has advantages over maintaining model diversity. However, solving SR_{MOP} is time-consuming because of the ergodic impossibility of the domain $\mathbb{F}$ and the difficulties in improving symbolic models, which often result in blind search and slow convergence for solution algorithms.

$$(\text{SR}_{\text{MOP}}) \begin{cases} \min & RMSE_{SR} = err(f_D, D) \\ \min & SIZE_{SR} = node(f_D) \\ s.t. & f_D \in \mathbb{F} \\ & SIZE_{SR} \in [\min_{size}, \max_{size}] \end{cases} \tag{4}$$

4. Decomposition for symbolic regression

4.1. Motivation

The idea of decomposition for symbolic regression is mainly motivated by our successful experiences in manually constructing symbolic models for practical data. This model-constructing procedure can be summarized as two steps: determining rough models and ameliorating models. The rough model that refers to the model form or type in this paper was determined according to the holistic observation on how y changes with $\mathbf{x}$ in historical data. We do this step via observing some representative data that globally distributed in the training data. This step usually requires strong professional background and extensive experiences in specifying a reasonable model form, especially for the complex data. In next step, the coefficients, constants and components of the rough model will be adjusted to improve the model accuracy by using detailed or partial data coupled with coefficient estimation methods. Such detailed or partial data are usually the sub-dataset with relatively large fitting error. In this paper, the procedures of determining rough models and ameliorating models are simulated by global regression (GR) and local regression (LR) respectively.

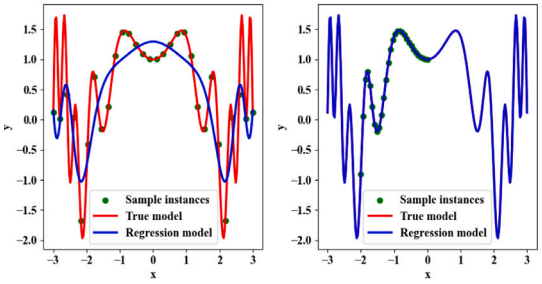
Above experiences make the authors aware that globally and locally sampled sub-datasets can play significant roles in distilling symbolic models for the given data, which can be vividly illustrated by Fig. 1. For the periodic data shown by Fig. 1(a), one can vividly find that the true model can be distilled by using least square fitting methods on the locally sampled sub-dataset. Fig. 1(b) vividly shows the advantages of GR over distilling the true model for noised data. Figs. 1(c) and 1(d) show the example of using the gplearn [20] to solve GR and LR for benchmarks Nguyen-6 [97] and Kattan-3 [164], which vividly show that GR (for Nguyen-6) or LR (for Kattan-3) can produce high-quality models. Figs. 1(c) and 1(d) also show that the models mix-trained by GR and LR based on gplearn becomes much better. High-quality models can be fast gained through fitting small sized sub-datasets that sampled from the original dataset. Different sub-datasets may lead to different model forms, which will be helpful in improving the model diversity for SR algorithms. This experience motivates us in designing this algorithmic framework for symbolic regression, which is shown by Fig. 2.

4.2. Global regression (GR) and local regression (LR)

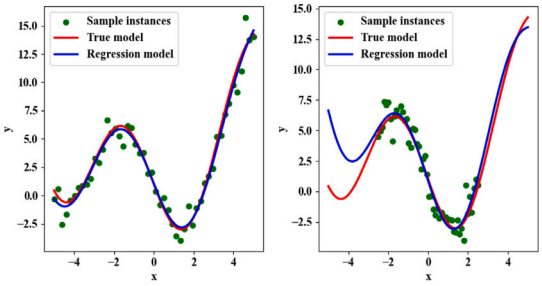
The global regression and local regression simulate the learning experiences of overviewing data in manual analysis to find the symbolic models that best fit the representative instances globally and locally sampled from the original dataset, aiming at determining the models to partially depict the global trends and detail (local) information. There are two main steps to construct a global or local regression task: data sampling and data regression. Let D^{sub} be the sets of instances sampled from D. The generalized subproblem Sub_{Prob} of determining symbolic models for D^{sub} can be stated by Eq. (5), which is formulated as the bi-objective model Sub_{MoP} shown by Eq. (6) in this paper.

$$(\text{Sub}_{\text{Prob}}) \quad f_{\text{sub}}^{\text{opt}} = \arg\min_{f_{\text{sub}} \in \mathbb{F}} \|f_{\text{sub}}(\mathbf{x}) - y\|_{(\mathbf{x},y)\in D^{\text{sub}}} \tag{5}$$

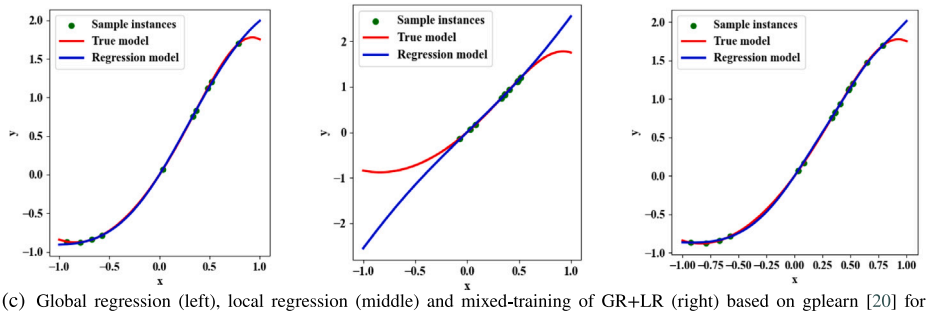
[1] https://scikit-learn.org/stable/index.html



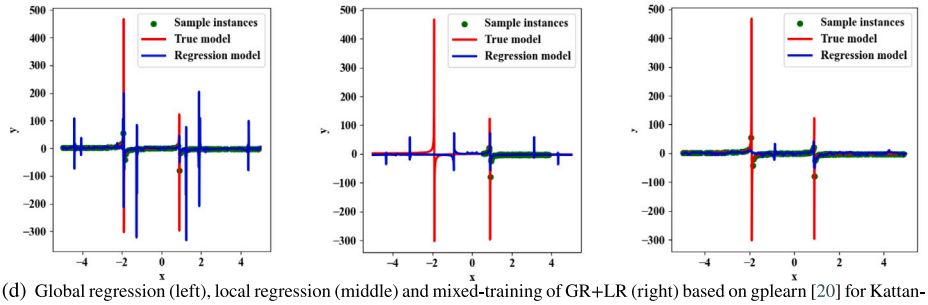
(a) GR (left) and LR (right) based on least square fitting for the periodic data generated by $y = \sin x^2 + \cos x^3$.



(b) GR (left) and LR (right) based on least square fitting for the noised aperiodic data generated by $y = e^{0.3x} - 5\sin x + 0.2x^2 + \Delta$, where $\Delta \sim N(0, 1)$.



(c) Global regression (left), local regression (middle) and mixed-training of GR+LR (right) based on gplearn [20] for Nguyen-6 [97]: $f(x) = \sin(x) + \sin(x^2 + x)$, $x \in [-1, 1]$.



(d) Global regression (left), local regression (middle) and mixed-training of GR+LR (right) based on gplearn [20] for Kattan-3 [164]: $f(x) = \frac{x}{1-\ln(x^2+x+1)}$, $x \in [-5, 5]$.

Fig. 1. Global regression (GR) and local regression (LR) for example datasets.

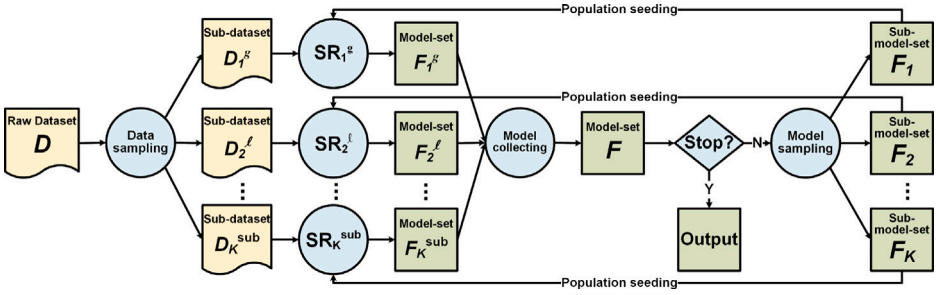


Fig. 2. The framework of decomposition based symbolic regression algorithm. D_i^g and D_j^ℓ denote the sub-dataset globally and locally sampled from D, SR_i^g and SR_i^ℓ the SR methods for D_i^g and D_j^ℓ, and F_i^g and F_j^ℓ the model sets obtained by SR_i^g and SR_i^ℓ respectively. D^{sub}, $\mathrm{SR}_i^{\mathrm{sub}}$ and F_i^{sub} should be the relevant notations for either global or local regression. F denotes the set of models gathered from all subtasks, and F_i the set of symbolic models sampled from F to seed the next generation population of $\mathrm{SR}_i^{\mathrm{sub}}$. (i,j=1~K).

$$(\mathrm{Sub}_{\mathrm{MoP}}) \begin{cases} \min & RMSE_{\mathrm{sub}} = err(f_{\mathrm{sub}}, D^{\mathrm{sub}}) \\ \min & SIZE_{\mathrm{sub}} = node(f_{\mathrm{sub}}) \\ s.t. & f_{\mathrm{sub}} \in \mathbb{F} \\ & D^{\mathrm{sub}} \subseteq D \\ & SIZE_{\mathrm{sub}} \in [\min_{size}, \max_{size}] \end{cases} \tag{6}$$

Let D^g and D^ℓ be the sets of instances globally and locally sampled from D respectively. The problems of global regression for D^g and local regression for D^ℓ can be stated by Eq. (6), and denoted as $\mathrm{GR}_{\mathrm{MOP}}$ and $\mathrm{LR}_{\mathrm{MOP}}$ respectively.

Many different sub-datasets with globally or locally sampled instances can be easily obtained through using different sampling methods. The globally representative sub-dataset in this paper is arbitrarily defined to be a set of instances picked at regular intervals from the original dataset according to the tuple index. The locally representative sub-dataset is defined to be a set of instances picked from a subrange of the tuple index. The volume of D^g or D^ℓ may be much smaller than that

of D, such that it may be good for the solution algorithm to obtain the accurate symbolic models for such sub-dataset, which will be helpful to improve other evolving models. It can be demonstrated vividly by Figs. 1(c) and 1(d) that the model obtained by gplearn on GR+LR is much better than both GR and LR. The evolving method for D^g or D^ℓ can be adopted as any symbolic regression algorithm. Detailed global regression method in this paper is constructed as Algorithm 1.

Algorithm 1: Global regression (GR) and Local regression (LR)

Input: The raw dataset D, number of subtasks K
Output: Subdataset for regression D^{sub}, the model set F^{sub}

$D^{\text{sub}} \leftarrow \emptyset$
Case Global regression:
  // Randomly select $\lfloor \frac{|D|}{K} \rfloor$ equispaced data instances
  // $\lfloor \frac{|D|}{K} \rfloor$ denotes the integer part of $\frac{|D|}{K}$
  $i_0 \leftarrow \texttt{randint}(1, \lfloor \frac{|D|}{K} \rfloor)$
  $k \leftarrow \texttt{randint}(2, K+1)$
  $i \leftarrow 1$
  while $i_0 + i * k < |D|$ **do**
    $D^{\text{sub}} \leftarrow D^{\text{sub}} \cup \{(\mathbf{x}_{i_0+i*k}, y_{i_0+i*k})\}$
    $i \leftarrow i + 1$
Case Local regression:
  // Randomly select $\lfloor \frac{|D|}{K} \rfloor$ contiguous data instances
  $i_0 \leftarrow \texttt{randint}(1, |D| - \lfloor \frac{|D|}{K} \rfloor)$
  for $i = i_0 \rightarrow i_0 + \lfloor \frac{|D|}{K} \rfloor$ **do**
    $D^{\text{sub}} \leftarrow D^{\text{sub}} \cup \{(\mathbf{x}_i, y_i)\}$
$F^{\text{sub}} \leftarrow \text{SR}^{\text{sub}}(D^{\text{sub}})$
return D^{sub}, F^{sub}

Function $\text{SR}^{\text{sub}}(D^{\text{sub}})$:
  $F^{\text{sub}} \leftarrow \emptyset$
  Option *1*:
    Initialize the form set $\Theta = \{\theta(\mathbf{x};\mathbf{a}) | \mathbf{a} = (a_0, a_1, \cdots) \in \mathbb{R}^{|\mathbf{a}|\times|\mathbf{a}|}\}$
      // $\mathbf{x}$ is the variable vector, $\mathbf{a}$ the parameter vector, and $\mathbb{R}$ the set of real numbers
    foreach $\theta(\mathbf{x};\mathbf{a}) \in \Theta$ **do**
      Use the least square method to determine $\mathbf{a}$ according to D^{sub}
      $F^{\text{sub}} \leftarrow F^{\text{sub}} \cup \{\theta(\mathbf{x};\mathbf{a})\}$
  Option *2*:
    Initialize the SR algorithm set $ALG = \{\texttt{alg}_1(D^{\text{sub}}), \texttt{alg}_2(D^{\text{sub}}), \cdots\}$
    for $i = 1 \rightarrow |ALG|$ **do**
      $F_i^{\text{sub}} \leftarrow \texttt{alg}_i(D^{\text{sub}})$
      $F^{\text{sub}} \leftarrow F^{\text{sub}} \cup F_i^{\text{sub}}$
  return F^{sub}

5. Decomposition based MoGP for symbolic regression

5.1. Encoding and genetic operators

5.1.1. Encoding

Model encoding has great impacts on the search efficiency for GP-based SR algorithms. Efforts [14,28,165] devoted to model encoding can be divided into two categories: tree-based representation [14,165] and sequence-based representation [28]. Improper coding for symbolic models in GP may lead to invalid offspring models and extra computation on modifying the models to ensure the validity. In this paper, binary trees are employed to represent the evolving symbolic models due to the advantages on: (1) each element can be expressed separately; (2) easy to decode; (3) each subtree is a feasible symbolic model.

5.1.2. Genetic operators

The main operators include crossover, mutation, selection and preservation.

Crossover. In GP, the crossover operator plays significant roles in exploiting the model space through recombining the genes of the selected parents to produce offspring models. Infeasible symbolic models produced in the search process will make the crossover operator inefficient. Hence, an important issue needed to be addressed is how to ensure the crossover operator always produce feasible offspring. Since each subtree of binary tree is a feasible sub-model, the crossover offspring in this paper is produced through exchanging the subtrees of selected parents, such that each produced binary tree can be ensured to be a feasible model. Let $f_1(x,y) = \cos(\frac{y}{x-6})$ and $f_2(x,y) = \sin(\frac{y}{-1.6+3x})$ be the parents selected for crossover, and their expression trees are shown in Fig. 3(a). Two expression trees can be produced by exchanging the subtrees in the circles, which can be decoded as $f_1'(x,y) = \cos(\frac{y}{3x})$ and $f_2'(x,y) = \sin(\frac{y}{-1.6+(x-6)})$.

Mutation. The mutation operator in the proposed algorithm is designed to include node mutation and form mutation aiming at improving the population diversity. In the binary tree, the leaf node saves a variable or constant, and the branch node saves an operator. The **node mutation** is developed to mutate the element in each selected node to other element with the same property, in which the selected leaf node should be mutated to another variable or constant, and the selected branch node should be mutated to another operator that can deal with the same operands. As shown in Fig. 3(b), the expression $\sin(\frac{x}{-4+x+y})$ is mutated to $\sin(\frac{y}{-4+x*y})$ via node mutation. In the node mutation, an important issue is how to update the node with the constant or coefficient, which has significant impact on the performance of the algorithm but, difficult to be addressed. In this paper, the node mutation for such nodes is defined to be randomly replaced by any variable from the variable set or floating point number from constant terminals (the terminals are unified as $[-10, 10]$) [166]. The **form mutation** is developed to exchange any two selected disjoint subtrees aiming at altering the model form of the parent model. As the example shown in Fig. 3(b), the offspring $\sin(\frac{x*y}{-4+x})$ can be obtained via using form mutation on the parent expression $\sin(\frac{x}{-4+x*y})$. Therefore, the population diversity can be improved.

Selection and preservation. In this paper, the fast non-dominated sorting approach and the selection operator are the same as that of NSGA-II [167]. To ensure the convergence of the algorithm, the elitist strategy is employed to preserve the next population.

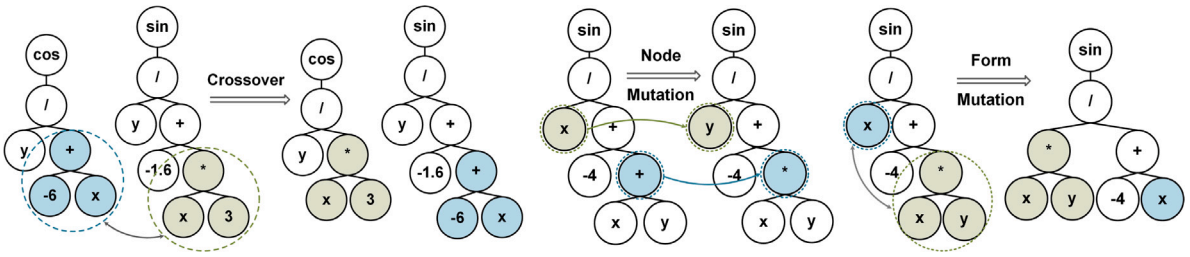
5.1.3. Hybrid MoGP for symbolic regression

Using the above operators and strategies, the developed MoGP-based SR algorithm is constructed as Algorithm 2, which can be used to solve both SR_{MOP} and Sub_{MoP}.

In the constructed algorithm, the algorithm parameters are adopted according to the guidance in [168,169]. The symbolic models that obtained by traditional regression methods can be introduced into the initial population or the evolving population during the evolutionary process when the population quality needs to be improved. This process in Algorithm 2 is called population disturbance, which will be executed at the certain generations. In this paper, the population disturbance is executed when the generation number t satisfies $t\%50 = 30$. Algorithm 2 will terminate when either the minimal RMSE of the evolving population or the total model evolving generation number (denoted as $TotalGen$) were reached. The TraReg in Algorithm 2 is suggested to be adopted as a set of traditional regression methods, e.g., polynomial regression, linear regression and nonlinear regression.

5.2. The proposed framework for decomposition based hybrid symbolic regression

The framework of the proposed decomposition based symbolic regression algorithm is a loop of the MoGP algorithms concurrently solving several subtasks coupled with a strategy of population redistribution, as illustrated by Fig. 2, in which the main steps include initialization, task decomposition, regression analysis, model collection



(a) The crossover operator. (b) The node mutation and form mutation.

Fig. 3. Crossover and mutation operators for genetic programming.

Algorithm 2: MoGP with traditional regression (MoGP+)

Input: Dataset D, population size pop_{size}, population POP, crossover probability p_c, mutation probability p_m, total generation $TotalGen$
Output: The nondominated models found so far

```
if POP = ∅ then
    randomly generate pop_size individuals for POP_0;
else
    POP_0 ← POP;
t ← 0;
while stopping criterion is not satisfied do
    Evaluation: POP_t ←evaluate(POP_t,D);
    O_c, O_m ← ∅;
    for i = 1 → pop_size/2 do
        if rand <= p_c then
            Selection: [P_1, P_2] ←select(POP_t);
            Crossover: [off_1, off_2] ←crossover(P_1, P_2);
            O_c ← O_c ∪ {off_1, off_2};
        if rand <= p_m then
            Selection: [Q_1, Q_2] ←select(POP_t);
            Node Mutation: q_1 ←NodMut(Q_1);
            Form Mutation: q_2 ←ForMut(Q_2);
            O_m ← O_m ∪ {q_1, q_2};
    O_t ← O_c ∪ O_m;
    if population disturbance is needed then
        D_temp ← random_select(D);
        O_tra ← TraReg(D_temp)// TraReg is the same as Option
            1 of Algorithm 1.
        O_t ← O_t ∪ O_tra;
    Evaluation: O_t ←evaluate(O_t,D);
    Preservation: POP_{t+1} ←elitist(POP_t ∪ O_t);
    t ← t + 1;
return The nondominated models found so far;
```

and redistribution. The designed algorithm framework in this paper is shown as Algorithm 3. In this paper, each subtask in the developed framework utilizes the same MoGP with the parameters adopted according to the relevant guidance in [166,168–170]. It is needed to adopt the total number of subtasks K, the size of the training dataset for each subtask, and the stopping criteria for both framework and subtask.

The number of subtasks is adopted according to the number of processor cores and the size of training dataset. The SR problem with small-sized training dataset (the number of training instances is set to be less than 50 in this paper) is decomposed into no more than two subtasks. For SR problem with large-sized training dataset, the number of subtasks is set to be less than the number of processor cores. In this paper, since the experiments are conducted by using the processor with 8 cores, we adopt the number of subtasks for large-sized problems to be 2, 4 and 6 respectively to test the influence of this parameter. The size of the training dataset for each subtask is arbitrarily set to be $\frac{|D|}{K}$ such that each training instance could be selected to train models.

Algorithm 3 will stop when either the minimal RMSE of the population or the maximal loop number (denoted as $MaxLoop$) were reached. The MoGP (adopted as Algorithm 2 in this paper) for each subtask will be suspended when the maximal generation (denoted as $MaxGen$) were reached. In this paper, the total number of model evolving generations (denoted as $TotalGen$) is equally divided and allocated to subtasks according to $MaxLoop$, $MaxGen$ and the subtask number K, which satisfies Eq. (7) for the sake of fair comparisons with other methods.

$$TotalGen = MaxLoop \cdot MaxGen \cdot K \tag{7}$$

At the end of each loop of Algorithm 3, the elite models obtained by the K subtasks will be gathered to the next population, and randomly distributed to the subtask for further evolved and improved. Note that the local regression and global regression in the subtask can be solved using any regression algorithm for the complex datasets.

6. Empirical evaluations and comparisons

6.1. Experimental settings

Ablation and comparison studies were conducted in Ubuntu 20.04 OS with an Intel Xeon E5-2620(v4) CPU to test the performance of the developed SR algorithms.

6.1.1. Ablation experiments

In the ablation experiments, nine benchmarks listed in Table 1 are used to evaluate the algorithm performance and make comparisons with several state-of-the-art methods in terms of model evolving behavior, mutual improvements between traditional regression methods and GP, and the algorithm performance. The algorithm performance was evaluated via the data depicting abilities (training RMSE and test RMSE), conciseness of the obtained models, and time efficiency.

Benchmarks These benchmarks are divided into three groups and, each contains three benchmarks. The benchmarks with simple forms that suitable for traditional regression methods are categorized as Group I, which will be used to test the mutual improvement between the traditional regression methods and genetic programming. Group II contains multivariate benchmarks with large training set aiming at evaluating the algorithm performance. Common used datasets taken from the UCI machine learning repository [173] are employed in Group III to evaluate the algorithm performance on solving real problems, which are randomly split with 90% of the data for training and the rest 10% for test in the 50 independent runs.

Table 1
Benchmarks for ablation experiments. U$[a,b,c]$ denotes the set of c random points sampled uniformly in $[a,b]$ for the variable, and E$[\alpha,\beta,\gamma]$ the grid of points evenly spaced with the interval of γ in $[\alpha,\beta]$. Test and training sets are independent.

	Function	Expression	Training set	Test set
Group-I	Nguyen-4 [97]	$\sum_{i=1}^{6} x^i$	x:U[−1,1,20]	x:U[−1,1,10]
	Nguyen-10 [97]	$2\sin(x\cos(y))$	x,y:E[−3,7,0.1]	x,y:E[7,8,0.1]
	Keijzer-15 [171]	$x^3/5 + y^3/2 - x - y$	x,y:E[−3,7,0.1]	x,y:E[7,8,0.1]
Group-II	Keijzer-10 [171]	x^y	x,y:U[0,1,100]	x,y:E[0,1,0.01]
	Vladislavleva-4 [38]	$\frac{10}{5+\sum_{i=1}^{5}(x_i-3)^2}$	x_1~x_5: U[0.05,6.05,1024]	x_1~x_5: U[0.05,6.05,5000]
	Korns-11 [172]	$6.87 + 11\cos(7.23x_1^3)$	x_1~x_5: U[−50,50,10000]	x_1~x_5: U[−50,50,10000]
Group-III	Abbreviation	Name	#Features	#Data(Train:Test)
	yac [173]	Yacht Hydrodynamics	7	308 (277:31)
	con [173]	Concrete Compressive	9	1030 (927:103)
	wineW [173]	Wine Quality-white	12	4898 (4408:490)

Algorithm 3: Decomposition based Genetic Programming(DGP+)

Input: Initial dataset D, population size pop_{size}, crossover probability p_c, mutation probability p_m, subtask number K, maximal generation $MaxGen$, maximal loop $MaxLoop$, SR algorithm set $ALG = \{\mathtt{alg}_1(D), \mathtt{alg}_2(D), \cdots\}$, model set $F = \{f | f \in \mathbb{F}\}$, and form set $\Theta = \{\theta(\mathbf{x};\mathbf{a}) | \mathbf{a} = (a_0, a_1, \cdots) \in \mathbb{R}^{|\mathbf{a}|\times|\mathbf{a}|}\}$;
Output: The nondominated models found so far
$POP_0 \leftarrow \emptyset$;
Initialization: randomly generate pop_{size} individuals for POP_0;
repeat
  $D^{\text{sub}} \leftarrow$`Random_Sample`$(D)$;
  for $j = 1 \rightarrow |ALG|$ **do**
    $f_j \leftarrow \mathtt{alg}_j(D^{\text{sub}})$;
    $POP_0 \leftarrow POP_0 \cup \{f_j\}$;
until $pop_{size}/2$ *models were produced*;
$POP_0 \leftarrow$`evaluate`(POP_0, D);
$t \leftarrow 0$;
Create K concurrent threads;
while $t < MaxLoop$ **do**
  for i=1→K **do** // `Deploy a subtask (GR or LR) to the` i^{th} `thread`
    $D_i^{\text{sub}} \leftarrow$`Sample`$(D)$// `Global sample for GR, local sample for LR.`
    $POP_i^{\text{sub}} \leftarrow$`random_subset`$(POP_t)$;
    foreach $\theta(\mathbf{x};\mathbf{a}) \in \Theta$ **do** // `Feed subpopulation.`
      Use the least square method to determine $\mathbf{a}$ according to D_i^{sub};
      $POP_i^{\text{sub}} \leftarrow POP_i^{\text{sub}} \cup \{\theta(\mathbf{x};\mathbf{a})\}$;
    $O_i \leftarrow$`MoGP`$(D_i^{\text{sub}}, pop_{size}, POP_i^{\text{sub}}, p_c, p_m, MaxGen)$ // `This paper uses Algorithm 2` $O_i \leftarrow$`evaluate`(O_i, D);
  Synchronize: `syncthreads`();
  $POP_{t+1} \leftarrow$`elitist`$(\bigcup_{i=1}^{k} O_i)$;
  $t \leftarrow t + 1$;
Release threads;
return The nondominated models found so far;

Compared methods The compared methods in ablation experiments are selected or constructed as follows:

- **TraReg** refers to the best results obtained by the traditional regression methods used in the developed algorithms. In this paper, the preset forms used in TraReg of the developed algorithms are shown in Table 2, and the coefficients will be addressed by the ordinary least squares method. To evaluate the mutual improvements between traditional methods and GP, the traditional regression methods are employed to determine the symbolic models with one or two randomly selected variables for the multivariable benchmarks in the experiments, named as "incomplete" models. In this way, various "incomplete" models will be produced for further evolving in GP.
- **GB** refers to the gradient boost based regression algorithm [174] taken from the `scikit-learn` library in Python.
- **gplearn**[2] refers to the GP implemented in Python with a scikit-learn inspired and compatible API [20]. The constant terminal is [−10,10].
- **EFS**[3] refers to the Evolutionary Feature Synthesis [175].
- **GSGP** refers to Geometric Semantic Genetic Programming [11], which treats the symbolic regression as a single objective optimization problem and works by expanding the models using the geometric semantic operators to determine the models to best fit the training data. The constant terminal is [−1,1].
- **GSGP-GD**[4] refers to the GSGP based on geometric dispersion (GD) [10]. The constant terminal is [−1,1].
- **GSGP-Red**[5] refers to the GSGP based on reduced trees [9]. The constant terminal is [−1,1].
- **ML-PIE**[6] refers to the GP based approach named Model Learning with Personalized Interpretability Estimation [19] that implemented in Python. The constant terminal is [−5,5].
- **MoGP** refers to Algorithm 2 without traditional regression that implemented in Python.
- **MoGP**+ refers to Algorithm 2 implemented in Python.
- **DGP**-k refers to Algorithm 3 without traditional regression methods that implemented in Python, in which each of the k concurrent subtasks was solved by the **MoGP**.
- **DGP**-k+ refers to Algorithm 3 with k concurrent subtasks that implemented in Python, in which the **MoGP**+ was employed as the SR algorithm.

Table 2
The preset forms of TraReg used in MoGP+ and DGP+.

Form	Variables	Coefficients
$f(x) = ax + b$	x	a, b
$f(x) = a + b\zeta(kx)$	x	a, b, k
$f(x) = a + bx^k$	x	a, b, k
$f(x) = \sum_{i=0}^{6} a_i x^i$	x	$a_0, \ldots, a_6$
$f(\mathbf{x}) = a + bx_1 + cx_2$	$\mathbf{x} = (x_1, x_2)$	a, b, c
$f(\mathbf{x}) = a_0 + \sum_{i=1}^{2} a_i \xi(k_i x_i)$	$\mathbf{x} = (x_1, x_2)$	a, b, k_1, k_2
$f(\mathbf{x}) = a_0 + \sum_{i=1}^{2}(\sum_{j=1}^{3} a_{ij} x_i^j)$	$\mathbf{x} = (x_1, x_2)$	$a_0,\ a_{ij}$

$\zeta(\cdot)$ refers to $\sin(\cdot)$, $\cos(\cdot)$ or $\ln(\cdot)$; $\xi(\cdot)$ refers to $\sin(\cdot)$ or $\cos(\cdot)$.

[2] https://github.com/trevorstephens/gplearn
[3] http://exgp.github.io/efs
[4] https://github.com/luizvbo/gsgp-gd
[5] https://github.com/laic-ufmg/GSGP-Red
[6] https://github.com/marcovirgolin/pyNSGP

Table 3
Parameter settings for MoGPs and DGPs.

	Parameter	Value
General	Population size (pop_{size})	300
	Stopping criteria:	
	min(POP.RMSE)	<1.0e−16
	TotalGen	1800(#Training≥50) 600(#Training<50)
Specific	Stopping criteria:	
	MaxGen (for DGPs)	50
	MaxLoop(for DGPs)	TotalGen/(MaxGen·K)
	Crossover rate (p_c) [168]	0.7
	Mutation rate (p_m)	0.1
	Model size (min-max)	3-50
	Tree depth (min-max)	2-8
	Number of sampling data	$\|D\|/K$
	Subtask number (K)	2\4\6
	Constant terminal	[−10,10]
	Operators	{+, −, ×, ÷, sin, cos, POW, log }

The general parameters for compared non-traditional methods are adopted as that in Table 3, and the other algorithm parameters are the same as that in the relevant literatures. The parameters for the developed MoGPs and DGPs are listed in Table 3. In experiments, each method was executed 50 times independently. The running times of the methods implemented in Python were compared.

6.1.2. Comparison experiments

We conducted the comparison experiments using the SRBench with the same settings as that in [25]. As an open-source, reproducible benchmarking platform for symbolic regression, SRBench[7] uses a wide range of diverse regression problems including both 122 black-box regression problems (available at PMLB[8]) with no known model form as well as 130 ground-truth benchmark problems (including 116 Feynman[9] datasets and 14 Strogatz[10] datasets) with known, ground-truth model forms to comprehensively assess the SR methods via various perspectives [25]. The Feynman datasets are used to test the abilities of the methods in distilling symbolic models for static systems and, the Strogatz problems for nonlinear and chaotic dynamical processes [176]. The datasets used for comparison studies in SRBench are with 47∼10^6 samples and 2∼10^3 features and, their properties are vividly shown by Fig. 4 in regard to the distribution of dataset sizes by samples and features. In experiments, white gaussian noise with different noise levels was added to the target data of the ground-truth problems, which was produced according to the signal root mean square value [25]. The target noised data with level $\gamma \in \{0.0, 0.001, 0.01, 0.1\}$ was calculated by Eq. (8).

$$y_{noise} = y + \delta, \quad \delta \sim N\left(0, \gamma\sqrt{\frac{1}{|D|}\sum_{|D|} y_i^2}\right) \tag{8}$$

In comparisons, the metrics including accuracy, complexity and solution criteria were employed, and results are reported by Figs. 11–12. In SRBench, the accuracy was assessed using the R^2 test, which is the coefficient of determination defined as Eq. (9). The complexity of the models returned by each method was calculated after simplifying via `sympy`.[11] For the ground-truth regression problems, the Symbolic Solution [25] metric was used for evaluating the ability of an SR method to discover the data generating process.

$$R^2 = 1 - \frac{\sum_i (y_i - \tilde{y}_i)^2}{\sum_i (y_i - \bar{y}_i)^2} \tag{9}$$

[7] https://github.com/cavalab/srbench
[8] https://epistasislab.github.io/pmlb/
[9] https://space.mit.edu/home/tegmark/aifeynman.html
[10] https://github.com/lacava/ode-strogatz
[11] https://www.sympy.org/en/index.html

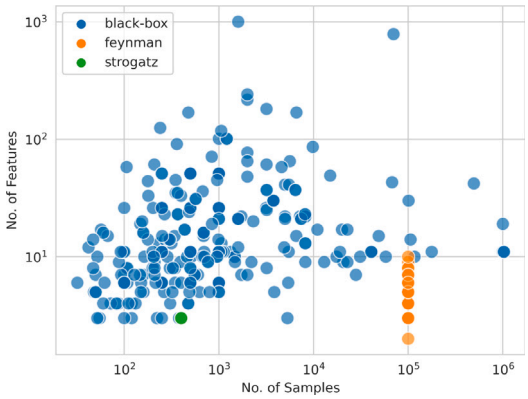


Fig. 4. Size distribution of datasets for comparison experiments [25].

6.2. Results and discussions

The experimental results for evaluating the proposed symbolic regression algorithm are reported mainly in the form of figures. Some detailed results will be listed in tables for accurate evaluations and comparisons on the algorithms. According to the Pareto theory of multiobjective optimization, there will be a set of nondominated symbolic models for SR_{MOP}. The symbolic models with the minimal training RMSE of each run will be selected for further analysis and comparisons. The developed algorithms will be evaluated in the respects of mutual improvements between traditional methods and GP, overall performance on different problems, and the behavior on evolving symbolic models. The convergence behavior of the developed algorithms is shown by Fig. 5. The results are shown in Figs. 7–10, and some supplementary detailed results are listed in Tables 4 and 5.

6.2.1. Ablation studies

The algorithm performance will be evaluated via ablation experiments and comparisons in terms of model evolving behavior, model accuracy, model conciseness and time efficiency. Since some valuable experimental results and the differences between the methods for some benchmarks (e.g., Keijzer-15, Korns-11 and con) cannot be vividly shown by figures, supplementary detailed results are given in Table 5.

Model evolving behavior The algorithm convergency and diversity of elite models are observed to show the model evolving behavior of the developed methods. The convergency of the developed methods was evaluated in experiments by observing how the averaged best RMSE changes with model evaluations for MoGP, DGPs, MoGP+ and DGP+s across 50 independent runs on five representative benchmarks with different number of variables, and the results are shown by Fig. 5. The advantages of coupling traditional regression methods into GPs can be found vividly from the subfigures in the upper line of Fig. 5, in which the curves of the training RMSEs with model evaluations for GP+s are under GPs with quite obvious gaps. To distill the models from training data for these five benchmarks, GP+s start from quite better solutions and evolve them to even better ones. DGP+s converge a bit slower than MoGP+ due to the models in DGP+s are evolved with smaller datasets. The subfigures in lower line of Fig. 5 show the abilities of the evolving symbolic models on depicting the test data, the convergence trends are quite similar to that on the training data except con. For the real-world dataset con, the data depicting abilities of the evolving models on the training and test dataset are quite different from the other benchmarks. The models obtained by MoGP+ and DGP+s are with much better training RMSEs but significantly worse test RMSEs than that of MoGP and DGPs. What is more, the convergent tendencies of training RMSEs are not held for test RMSEs. This phenomenon seems to happen in solving Nguyen-10 by DGP-2. Such phenomenon

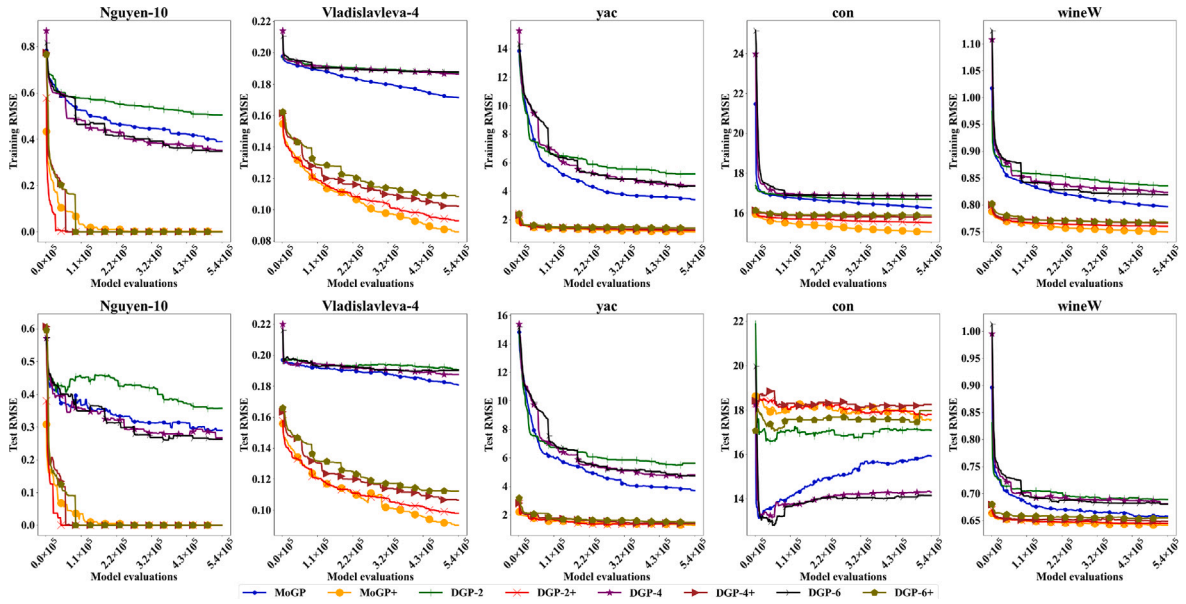


Fig. 5. The model evolving behaviors of MoGP, MoGP+, DGPs and DGP+s. The curves denote the averaged RMSE values across 50 independent runs.

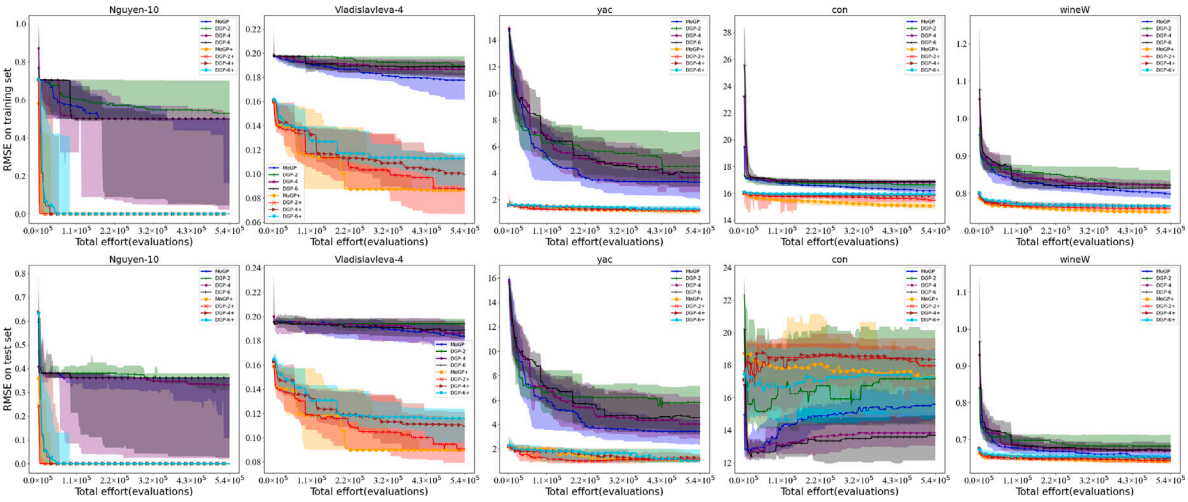


Fig. 6. RMSEs of non-dominated elite models evolved by MoGP, MoGP+, DGPs and DGP+s. The solid lines and shaded areas with the same color scheme denote the median values and distribution regions of the RMSEs of non-dominated elite models obtained by relevant algorithms.

demonstrates that the models distilled from the training data with high RMSEs may not depict the true laws in the data well, which may be caused by overtraining. The diversity was evaluated by the RMSE distribution of the non-dominated elite models during the evolution processes of MoGP, DGPs, MoGP+ and DGP+s, which are shown in Fig. 6. The convergent tendency of median RMSEs in Fig. 6 are similar to that of average RMSEs in Fig. 5. The distribution regions of the results show that there are diverse non-dominated elite models during the evolution processes of the developed methods, which indicates the proposed framework has positive effect in reducing diversity loss.

Mutual improvements between traditional regression methods and GP This aspect is mainly discussed via the experiments on the benchmarks of Group I to show the performance of reducing blind search by traditional regression, in which Nguyen-4 [97] and Keijzer-15 [171] are polynomial problems, and Nguyen-10 [97] a mixed trigonometric problem. The detailed results for accurate comparisons are reported in Tables 4, which can vividly reveal the mutual improvements between traditional regression methods and GP.

The results in Figs. 7–8 and Table 4 show that GB and TraReg can obtain high precision models for the unary polynomial problem Nguyen-4 and, lower precision models for Nguyen-10 and Keijzer-15. The median and average RMSEs on both training and test data show that, without traditional regression methods, MoGP and DGPs perform much worse than GB and TraReg in solving Nguyen-4 and, almost the same in solving Nguyen-10 and Keijzer-15. Coupled with TraReg, MoGP+ and DGP+s can obtain higher precision models than GB, TraReg, MoGP and DGPs. Fig. 8 and Table 4 demonstrate that the models obtained by MoGP+ and DGP+s are with significantly better test RMSEs than that of TraReg, GB and the other compared methods, which indicates that the models obtained by MoGP+ and DGP+s can better approximate the true models for the benchmarks of Group I. Hence, the experimental results show the mutual improvements between traditional regression methods and GP. The results in Fig. 5 show that TraReg can significantly improve the training RMSE and convergence speed, which indicates TraReg can reduce the blind search effect. Fig. 6 also shows that TraReg can reduce diversity loss when solving Vladislavleva-4.

Model accuracy on data depicting abilities Such abilities that can indicate the search efficiency of the solution algorithms are usually evaluated

Table 4
Detailed results on testing mutual improvement between traditional methods and GP.

Method		RMSE on training data Min\Median\Avg(±std)	Size Min\Median\Avg(±std)	RMSE on test data Min\Median\Avg(±std)
Nguyen-4	GB	2.26e−9	–	9.37e−1
	TraReg	1.11e−10	50	1.11e−10
	MoGP	4.30e−17\5.36e−2\7.52e−2(±6.89e−2)	7\20\22.52(±11.53)	5.34e−17\4.00e−1\5.52e−1(±4.35e−1)
	MoGP+	1.11e−12**1.11e−12\5.43e−10(±1.07e−9)**	27\37\37.2(±**3.61**)	1.63e−11**2.50e−11\8.73e−9(±1.74e−8)**
	DGP-2	4.30e−17\1.08e−1\1.24e−1(±1.2e−1)	5\12\14.18(±7.41)	5.34e−17\4.64e−1\8.08e−1(±1.43)
	DGP-2+	**2.32e−17\5.54e−13\4.36e−10(±9.61e−10)**	21\25\28.42(±7.51)	**0.00\1.21e−11\7.03e−9(±1.6e−8)**
Nguyen-10	GB	4.24e−2	–	5.06e−1
	TraReg	6.90e−1	11	7.10e−1
	MoGP	5.25e−17\5.00e−1\3.99e−1(±1.97e−1)	5\5\8.26(±5.73)	2.10e−17\3.61e−1\2.83e−1(±1.55e−1)
	MoGP+	5.25e−17**5.25e−17\9.22e−17(±1.36e−16)**	5\5**6.46(±3.55)**	2.10e−17**2.10e−17\7.63e−17(±1.71e−16)**
	DGP-2	1.46e−5\5.00e−1\4.64e−1(±1.53e−1)	5\5\6.84(±5.25)	1.06e−5\3.61e−1\3.6e−1(±1.99e−1)
	DGP-2+	**5.25e−17\5.25e−17\6.25e−17(±5.79e−17)**	5\5**5.36(±1.69)**	**2.10e−17\2.10e−17\2.8e−17(±3.59e−17)**
	DGP-4	5.25e−17\5.00e−1\4.64e−1(±2.47e−1)	4\8\8.6(±3.9)	2.10e−17\3.61e−1\4.16e−1(±3.19e−1)
	DGP-4+	5.25e−17**5.25e−17\5.75e−17(±2.59e−17)**	5**5\5.12(±0.47)**	2.10e−17**2.10e−17\2.58e−17(±2.11e−17)**
	DGP-6	5.25e−17\4.27e−6\1.18e−1(±2.3e−1)	4\9\10.72(±5)	2.10e−17\3.08e−6\8.68e−2(±2.05e−2)
	DGP-6+	5.25e−17**5.25e−17\8.83e−3(±5.6e−2)**	5**5\6.36(±2.55)**	2.10e−17**2.10e−17\6.3e−3**(±3.68e−2)
Keijzer-15	GB	1.10e−1	–	4.53e+3
	TraReg	5.19e+1	11	2.24e+2
	MoGP	4.68\1.29e+1\1.27e+1(±4.28)	6\26\26.52(±13.69)	7.61\5.45e+1\6.25e+1(±3.23e+1)
	MoGP+	**6.35e−9\3.93e−8\4.39e−8(±2.55e−8)**	19\33\33.66(±**7.76**)	**8.12e−9\1.18e−7\1.83e−7(±4.49e−7)**
	DGP-2	6.73\1.64e+1\1.61e+1(±2.3)	6\12\14.82(±8.55)	1.68e+1\8.88e+1\8.66e+1(±2.3e+1)
	DGP-2+	**8.61e−9\4.81e−8\7.73e−8(±6.12e−8)**	21\33\33.58(±**4.81**)	**1.71e−8\8.79e−8\2.9e−7(±2.7e−7)**
	DGP-4	7.25\1.69e+1\1.61e+1(±2.27)	6\14\15.46(±7.94)	2.57e+1\8.98e+1\8.13e+1(±2.56e+1)
	DGP-4+	**2.30e−9\4.45e−8\5.15e−8(±3.92e−8)**	27\33\33.22(±**3.28**)	**7.04e−9\6.29e−8\1.62e−7(±1.81e−7)**
	DGP-6	5.16\1.40e+1\1.31e+1(±3.66)	6\16\17.26(±6.73)	9.85\7.00e+1\6.92e+1(±3.01e+1)
	DGP-6+	**7.40e−9\5.02e−8\5.05e−8(±2.82e−8)**	19\33\33.66(±7.93)	**9.80e−9\1.23e−7\1.54e−7(±1.32e−7)**

The data in black bold denotes the better results obtained by TraReg aided GPs.

through the training and test RMSEs, which can reflect the abilities of distilled models on approximating the true laws in the data. The proposed methods are evaluated according to the experimental results reported in Figs. 7–8 and Tables 4–5. The obtained models with high accuracy can indicate the search efficiency of the solution algorithms. The results for the benchmarks of Group I vividly show that MoGP+ and DGP+s outperform the other methods in distilling symbolic models with limited model size which can depict both training and test with significantly better RMSEs. In solving the benchmarks of Group II, MoGP+ and DGP+s exhibit good performance. For Vladislavleva-4, GB performs the best, and EFS outperforms other swarm-based regression methods in depicting the training data. However, for depicting the test data of Vladislavleva-4, EFS performs the worst, MoGP+ performs the best, and DGP+s perform better than the rest swarm-based regression methods. For Korns-11, GB performs the worst, and the rest methods perform similar in depicting the data. The RMSEs on training and test data of the models obtained by MoGP+ and DGP+s are with slight differences with the best results for the real-world data of Group III. For yac and wineW, GB performs much better than the swarm-based methods on depicting both of the training and test data. However, for con, GB performs the worst. These results indicate that there are many chances for traditional methods to play positive roles in improving the performance of swarm-intelligent SR methods. From training and test RMSEs in Figs. 7–8 and Figs. 4–5, MoGP+ and DGP+s can distill better models to approximate the true laws than other methods, and perform much stable. Hence, the proposed algorithm framework has advantages over reducing blind search.

Model conciseness Concise models are beneficial to offer vital insight into the data and wide applications on the distilled laws. The results reported in Table 5 vividly show that the model bloat happened when using GSGP, GSGP-GD and GSGP-Red to determine the symbolic models for the benchmarks. Hence, the size of the models obtained by these three methods are not plotted in Fig. 9. When solving multivariate problems, e.g., Vladislavleva-4, Korns-11 and wineW, the models obtained by gplearn included only a single constant or variable, which cannot truly depict the laws in data. The results shown by Fig. 9 demonstrate that MoGP, DGPs, MoGP+ and DGP+s have obvious advantages over controlling the model bloat, and well-designed GPs are able to determine readable expressions to depict the concealed laws in complex data.

Time efficiency To fairly evaluate the time efficiency, gplearn, ML-PIE, MoGP, DGPs, MoGP+ and DGP+s that realized in the same programming language are selected for comparisons, in which gplearn deals with the symbolic regression problem as a single optimization problem. The results are shown by Fig. 10, from which one can vividly find that DGPs and DGP+s spend significantly less time than MoGP and MoGP+, especially for complex benchmarks, and the run time decreases with growth of subtasks. In view of the well performance of DGP+s on distilling good symbolic models for the benchmarks, the time efficiency can be significantly improved by the decomposition strategy.

6.3 Comparison analysis

Aided by SRBench [25], comparisons results on the ground-truth and black-box benchmark problems are shown by Figs. 11 and 12.

The comparison results on the ground-truth regression problems for the compared SR methods sorted by their median solution rate are summarized in Fig. 11, in which the results are further grouped by data source (Feynman or Strogatz). As shown by Fig. 11(a), the average performance of DGP+ performs the best among the compared SR methods in terms of solution rate and, only AIFeynman performs better than DPG+ when the noise levels below 0.001. This indicates DGP+ outperforms other SR methods in identifying exact solutions of the time. AIFeynman is also verified in [25] to be able to identify exact solutions 53% of the time, significantly better than the next two closest methods (DGP+ 33%, GP-GOMEA 27%) and nearly twice as often as GP-GOMEA. At noise levels above 0.01, DGP+ recovers exact solutions the most often among the SR methods. The performance of AIFeynman performs differs significantly in noise levels. DGP+ performs a bit worse than GP-GOMEA, SBP-GP, AFP_FE and EPLEX in R^2 test, but outperforms these methods obviously in simplified complexity. The results of grouped problems in Figs. 11(b)–11(d) show that these SR methods have better performance in solving Feynman problems. The performance of DGP+ ranks 7 in terms of accuracy solutions and 6

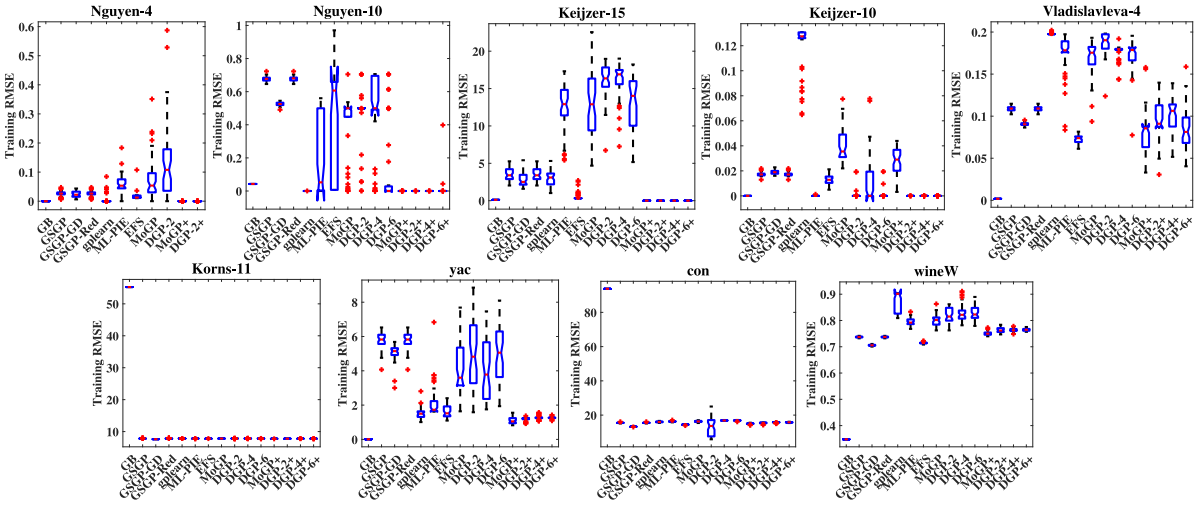


Fig. 7. Boxplots on the training RMSEs of the models obtained by the compared methods.

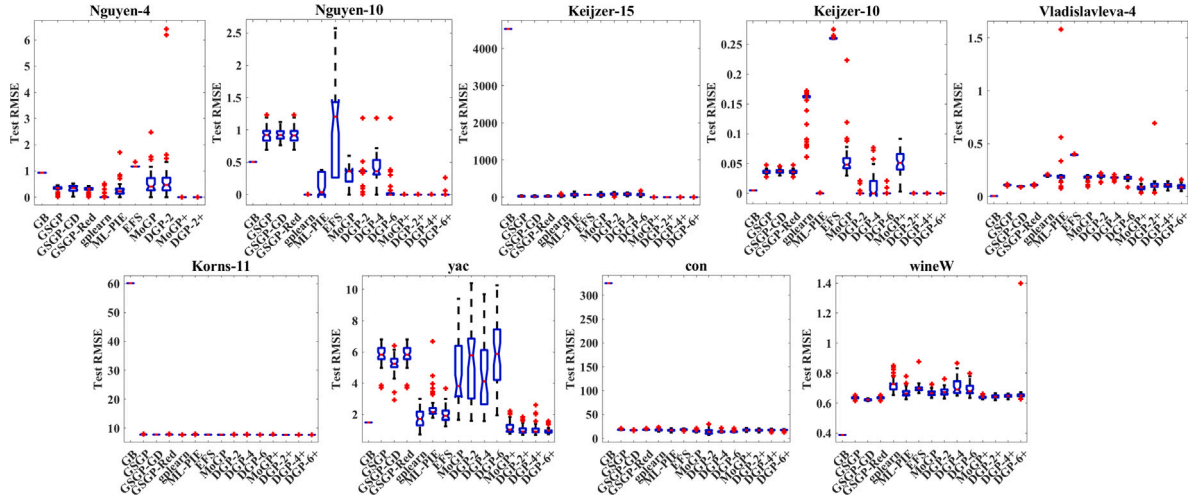


Fig. 8. Boxplots on the test RMSEs of the models obtained by the compared methods.

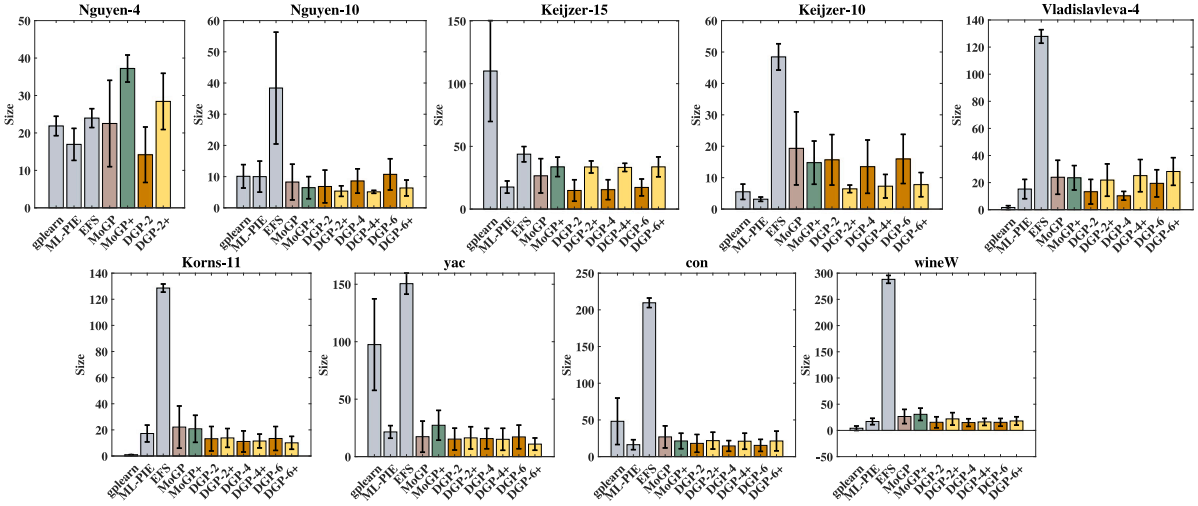


Fig. 9. Comparisons on the models sizes of the compared methods.

Table 5
Supplementary detailed results for performance evaluations on the selected benchmarks. The method with * denotes the best results of the relevant algorithm suites. The data in black bold denotes the best results, and red the abnormal result.

Method		RMSE on training data Min\Median\Avg(±std)	RMSE on test data Min\Median\Avg(±std)	Size Min\Median\Avg(±std)
Keijzer-15	GSGP*	1.59\2.51\2.9(±9.49e−1)	2.22\1.86e+1\2.19e+1(±1.29e+1)	−\−\−
	gplearn	1.02\3.10\2.96(±8.65e−1)	4.50\3.35e+1\4.04e+1(±2.23e+1)	45\101\110.06(±40.22)
	ML-PIE	5.42\1.29e+1\1.25e+1(±3.11e+1)	4.53\7.17e+1\6.49e+1(±3.18e+1)	7\18\17.56(±4.86)
	EFS	3.11e−1\3.27e−1\5.39e−1(±5.35e−1)	6.68e+1\6.68e+1\6.68e+1(±4.26e−14)	21\45\43.76(±6.10)
	MoGP	4.68\1.29e+1\1.27e+1(±4.28)	7.61\5.45e+1\6.25e+1(±3.23e+1)	**6**\26\26.52(±13.69)
	MoGP+	6.35e−9**3.93e−8****4.39e−8**(±**2.55e−8**)	**8.12e−9****1.18e−7**\1.83e−7(±4.49e−7)	19\32.5\33.66(±7.76)
	DGP-k*	5.16\1.40e+1\1.31e+1(±3.66)	9.85\7.00e+1\6.92e+1(±3.01e+1)	**6****11.5****14.82**(±**8.55**)
	DGP-k+*	**2.30e−9**\4.45e−8\5.05e−8(±2.82e−8)	7.04e−9\6.29e−8**1.54e−7**(±**1.32e−7**)	19\33\33.22(±3.28)
Keijzer-10	GSGP*	1.29e−2\1.71e−2\1.73e−2(±1.43e−2)	2.79e−2\3.57e−2\3.62e−2(±3.12e−3)	−\−\−
	gplearn	6.49e−2\1.28e−1\1.2e−1(±1.92e−2)	6.05e−2\1.62e−1\1.47e−1(±3.33e−2)	4\4\5.5(±2.44)
	ML-PIE	**3.85e−17****3.85e−17**\2.53e−5(±1.77e−4)	**2.84e−17****2.84e−17**\2.58e−5(±1.81e−4)	**3****3****3.14**(±**0.69**)
	EFS	4.94e−3\1.28e−2\1.28e−2(±3.71e−3)	2.59e−1\2.60e−1\2.61e−1(±2.41e−3)	41\49\48.44(±4.19)
	MoGP	2.20e−2\3.55e−2\3.91e−2(±1.15e−2)	2.97e−2\4.81e−2\5.5e−2(±2.92e−2)	7\15\19.32(±11.63)
	MoGP+	3.16e−3\2.90e−2\2.63e−2(±1.08e−2)	3.10e−3\5.11e−2\4.96e−2(±2.03e−2)	7\14\14.78(±6.88)
	DGP-k*	**3.68e−17****3.85e−17**\8.49e−4(±3.79e−3)	2.86e−17\2.86e−17\8.33e−4(±3.73e−3)	4\12\13.5(±8.49)
	DGP-k+*	3.85e−17**3.85e−17****9.23e−17**(±**1.14e−16**)	2.86e−17\2.86e−17**8.56e−17**(±**1.24e−16**)	**3**\6\6.44(±1.2)
Vladislavleva-4	GSGP*	8.65e−2\9.03e−2\9.04e−2(±1.72e−3)	8.74e−2\9.58e−2\9.56e−2(±2.59e−3)	−\−\−
	gplearn	1.97e−1\1.98e−1\1.98e−1(±8.21e−4)	1.85e−1\1.96e−1\1.96e−1(±4.34e−3)	***1******1******1.6(±1.39)***
	ML-PIE[a]	8.35e−2\1.78e−1\1.75(±2.38e−2)	7.55e−2\1.82e−1\2.27e−1(±2.19e−1)	**3**\16\15.22(±7.148)
	EFS	6.12e−2**7.39e−2****7.24e−2**(±**4.88e−3**)	3.87e−1\3.93e−1\3.94e−1(±2.97e−3)	109\129\127.86(±5.00)
	MoGP	9.36e−2\1.75e−1\1.69e−1(±1.97e−2)	9.08e−2\1.81e−1\1.78e−1(±1.99e−2)	6\21\23.92(±12.55)
	MoGP+	3.30e−2\8.56e−2\7.98e−2(±2.3e−2)	**3.61e−2**\8.92e−2**8.46e−2**(±**2.16e−2**)	9\21\23.56(±9.01)
	DGP-k*[b]	7.75e−2\1.79e−1\1.72e−1(±1.83e−2)	8.78e−2\1.78e−1\1.76e−1(±1.01e−2)	**3****9****10.26**(±**3.23**)
	DGP-k+*[c]	**3.06e−2**\8.13e−2\8.45e−2(±2.48e−2)	3.62e−2**8.69e−2**\9.03e−2(±2.24e−2)	8\15\21.86(±11.93)
Korns-11	GSGP*	**7.61****7.63****7.63**(±**6.41e−3**)	7.75\7.77\7.79(±4.81e−3)	−\−\−
	gplearn	7.83\7.83\7.83(±1.87e−5)	7.76\7.76\7.76(±2.01e−4)	***1******1******1(±0)***
	ML-PIE	7.76\7.82\7.81(±2.03e−2)	7.69\7.75\7.75(±4.43e−2)	**3**\19\17.22(±6.44)
	EFS	7.76\7.76\7.77(±1.53e−2)	7.73\7.74\7.75(±1.54e−2)	121\129\128.58(±3.11)
	MoGP	7.75\7.82\7.8(±2.96e−2)	7.68\7.76\7.74(±2.94e−2)	4\17\22.14(±16.15)
	MoGP+	7.72\7.75\7.75(±1.02e−2)	**7.65****7.69**\7.68(±1.68e−2)	5\17\20.82(±10.36)
	DGP-k*	7.77\7.83\7.82(±1.24e−2)	7.70\7.76\7.75(±1.19e−2)	4**8**\11.14(±8.08)
	DGP-k+*	7.75\7.77\7.77(±6.55e−3)	7.68\7.70**7.7**(±**7.69e−3**)	4\9**10.1**(±**4.94**)
wineW	GSGP*	**7.01e−1****7.06e−1****7.06e−1**(±**2.19e−3**)	**6.13e−1****6.23e−1****6.23e−1**(±**4.84e−3**)	−\−\−
	gplearn	8.09e−1\9.01e−1\8.71e−1(±3.81e−2)	6.54e−1\7.26e−1\7.24e−1(±4.08e−2)	***1******1****4.14(±4.29)*
	ML-PIE	7.68e−1\7.92e−1\7.94e−1(±1.45e−2)	6.25e−1\6.64e−1\6.70e−1(±2.74e−2)	6\18\17.04(±6.10)
	EFS	7.08e−1\7.14e−1\7.15e−1(±2.79e−3)	6.67e−1\6.98e−1\7.00e−1(±2.90e−2)	266\290\288.12(±7.54)
	MoGP	7.62e−1\8.02e−1\8.00e−1(±2.01e−2)	6.36e−1\6.66e−1\6.67e−1(±2.01e−2)	**4**\24\26.56(±13.55)
	MoGP+	7.40e−1\7.50e−1\7.51e−1(±7.00e−3)	6.25e−1\6.39e−1\6.40e−1(±8.05e−3)	7\31\30.7(±11.73)
	DGP-k*	7.62e−1\8.15e−1\8.20e−1(±2.55e−2)	6.31e−1\6.73e−1\6.74e−1(±2.31e−2)	**4****12****14.94**(±**7.32**)
	DGP-k+*	7.47e−1\7.62e−1\7.63e−1(±9.62e−3)	6.20e−1\6.43e−1\6.44e−1(±1.05e−2)	5\15\16.2(±6.73)
con	GSGP*	1.31e+1**1.32e+1****1.32e+1**(±**5.7e−2**)	1.58e+1\1.71e+1\1.7e+1(±5.22e−1)	−\−\−
	gplearn	1.55e+1\1.60e+1\1.6e+1(±3.04e−1)	1.40e+1\1.78e+1\1.77e+1(±2.03)	13\35\48.14(±31.67)
	ML-PIE	1.58e+1\1.63e+1\1.63e+1(±2.6e−1)	1.25e+1\1.70e+1\1.7e+1(±1.86)	5\16\16.28(±6.78)
	EFS	1.39e+1\1.44e+1\1.44e+1(±1.59e−1)	1.52e+1\1.88e+1\1.85e+1(±1.10)	183\211\209.6(±6.50)
	MoGP	1.54e+1\1.63e+1\1.63e+1(±3.78e−1)	1.26e+1\1.52e+1\1.56e+1(±1.89)	5\27\26.88(±15.02)
	MoGP+	1.42e+1\1.50e+1\1.5e+1(±3.41e−1)	1.33e+1\1.71e+1\1.77e+1(±2.20)	7\18\21.32(±10.57)
	DGP-k*	**5.87**\1.36e+1\1.35e+1(±5.57)	**7.38****1.36e+1****1.41e+1**(±**1.69**)	5**13****14.56**(±**7.29**)
	DGP-k+*	1.42e+1\1.54e+1\1.54e+1(±3.48e−1)	1.26e+1\1.71e+1\1.69e+1(±1.75)	**4**\14\21(±10.92)

The model bloat effect occurred for GSGP* due to the obtained model sizes are over 10^{46}.

[a] Succ(EFS)=46/50.

[b] Succ(DGP-2)=Succ(DGP-6)=49/50.

[c] Succ(DGP-6+)=49/50, where Succ(·) denotes the rate of successful runs.

for R^2 test. One can vividly note the obvious differences in the ranks according to these three metrics of some SR methods, such as Operon, MRGP and AIFeynman.

By comparing the results of each SR methods shown in Figs. 11(b)–11(d), we can find the obvious differences of their performance between Feynman and Strogatz problems. Almost all the SR methods perform better on Feynman problems than Strogatz problems. It is more difficult for the SR methods to solve the Strogatz problems than Feynman problems due to the 2-state dynamic models to be determined for the Strogatz problems should be not only accurate but also able to match the underlying processes of generating the data.

On the black-box benchmark problems, we show the experimental results via Fig. 12 to compare our algorithm with other methods. The results of the compared methods are that reported in [25]. The overall performance comparisons of the methods by considering both accuracy and simplicity simultaneously are summarized in the Pareto plot as Fig. 12(a). The median test set results of all methods in R^2 test and model size are shown respectively in Fig. 12(b).

There are four methods in the Pareto front of median rankings in R^2 test and model size as shown in Fig. 12(a): Operon, GP-GOMEA, DGP+ and DSR, which indicates that they can give the best trade-off sets between accuracy and simplicity for the black-box regression problems. From Fig. 12(b), one can vividly find that the mean of the median R^2 test of DGP+ ranks much better than the median rankings in Fig. 12(a), which indicates DGP+ has stable performance in finding

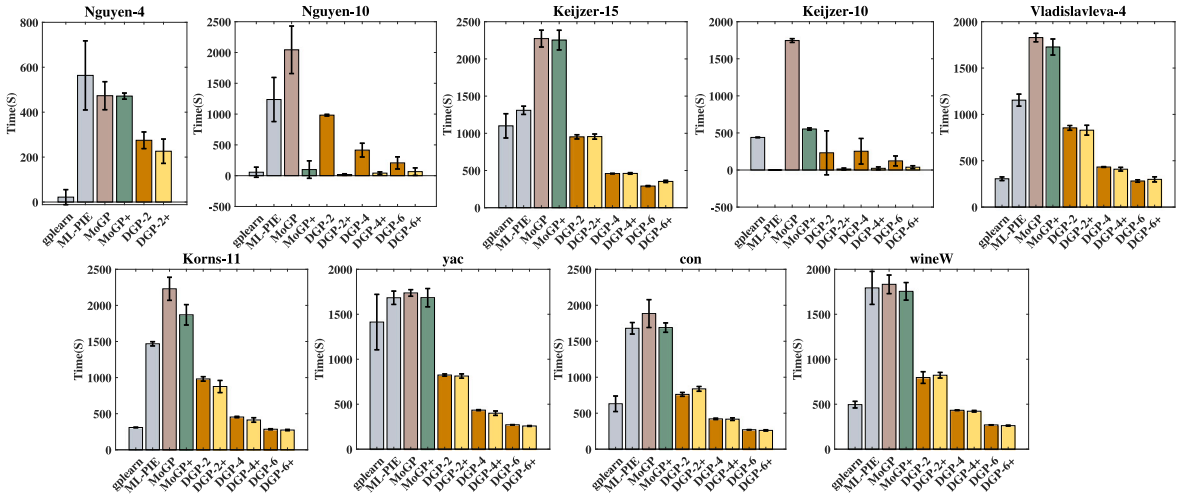
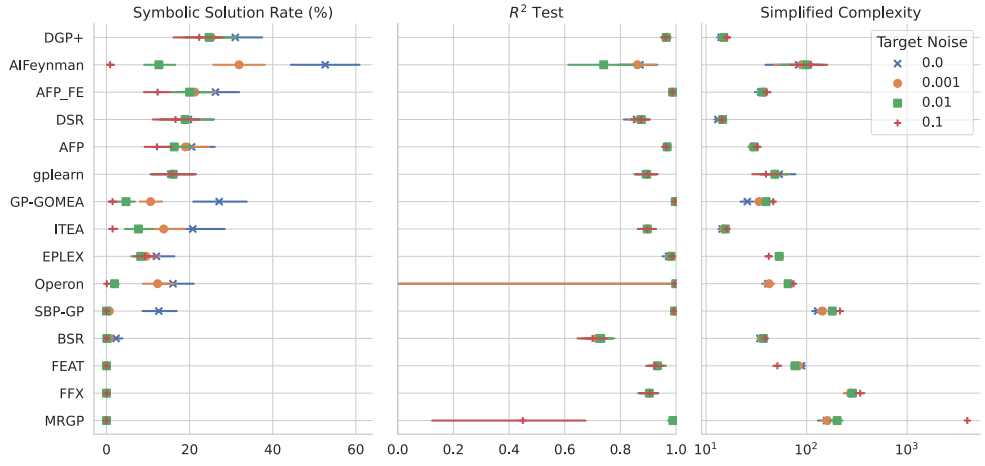
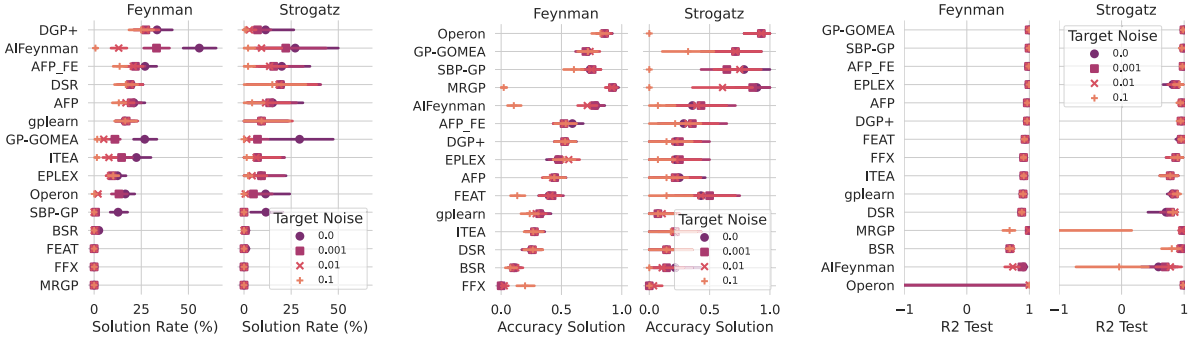


Fig. 10. Comparisons on the running times of the selected methods in solving the benchmarks.



(a) Results on the ground-truth regression problems.



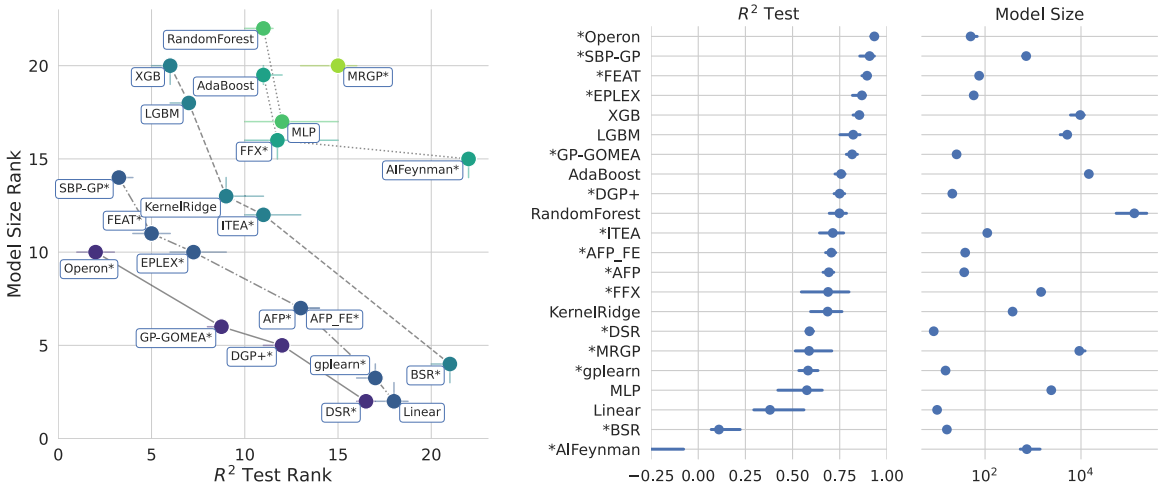
(b) Solution rates for grouped problems. (c) Accuracy solutions for grouped problems. (d) R^2 test scores for grouped problems.

Fig. 11. Comparisons results of the SR methods on the ground-truth regression problems differentiated by noise level in terms of solution rates, accuracy solutions (models with $R^2 > 0.999$ on the Feynman and Strogatz problems) and R^2 scores. The bars show the 95% confidence interval.

expressive symbolic models. As shown in Fig. 12(b), the models produced by five of the top ten compared methods (SBP-GP, XGB, LGBM, AdaBoost and RandomForest) are with significantly larger size than the rest five methods. Operon performs the best in finding expressive symbolic models. There are six GP-based SR methods among the top ten methods (Operon, SBP-GP, FEAT, EPLEX, GP-GOMEA, DGP+), which demonstrate that genetic programming is more suitable in finding interpretable models from data.

7 Conclusion

We developed a cross-parallel multiobjective genetic programming algorithm for symbolic regression coupled with a task decomposition strategy, by which the original regression problem is divided into a set of simple subproblems including global regression and local regression that are concurrently solved by regression methods. Each subproblem is defined to be a regression problem for a specific or representative subset of the original dataset. The global regression subproblem for



(a) Pareto plot comparing the rankings. Points denote median rankings. Connecting lines and color denote Pareto dominance rankings.

(b) Results on the black-box regression problems. Points indicate the mean of the median test set performance on all problems.

Fig. 12. Comparison of all methods on the black-box datasets in terms of R^2 test scores and model size. The bars show the 95% confidence interval and, methods marked with an asterisk are SR methods.

globally sampled data and local regression subproblem for locally sampled data aim to fast determine high quality symbolic models that be able to partially depict global and local information in the original data. The obtained global and local regression models will be seeded into the population and trained randomly by other subtasks to improve the model accuracy. To improve diversity, the population during evolution process were seeded by the models obtained by traditional regression methods for randomly sampled data. Ablation experiments and comparisons by SRBench are conducted to evaluate the performance and effectiveness of the proposed algorithm framework for symbolic regression. Results demonstrate the effectiveness of the algorithm in terms of the mutual improvements between traditional regression methods and genetic programming, algorithm convergency, model accuracy, bloat control and time efficiency. Comparison results show the stable performance of the developed algorithm on the wide range problems, especially the advantages over solution rate and model conciseness. However, ablation studies show the effects of overtraining or overfitting may occur for the developed algorithm, e.g. con, which will be investigated in the future research. The selection and collaboration of subtask algorithms and the recognition of important sub-datasets will be further studied.

CRediT authorship contribution statement

Lei Fan: Writing – original draft, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Zhaobing Su:** Software. **Xiyang Liu:** Writing – review & editing, Supervision. **Yuping Wang:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The authors would like to thank Mr. Zhaobing Su for his efforts on programming work while doing his M.S. thesis under the authors' supervision. The authors are also grateful to the anonymous reviewers for their valuable comments and suggestions. This work was supported by the National Natural Science Foundation of China (62272367, 82172860, 61502371), Joint Funds of National Natural Science Foundation of China (U22A2098) and Natural Science Basic Research Program of Shaanxi (Program No.2018JQ6047).

References

[1] L. Vanneschi, SLIM_GSGP: The non-bloating geometric semantic genetic programming, Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 14631 LNCS, Springer Nature Switzerland, 2024, pp. 125–141, http://dx.doi.org/10.1007/978-3-031-56957-9_8, URL https://doi.org/10.1007/978-3-031-56957-9_8.

[2] G. Nadizar, B. Sakallioglu, F. Garrow, S. Silva, L. Vanneschi, Geometric semantic GP with linear scaling: Darwinian versus Lamarckian evolution, Genet. Program. Evol. Mach. 25 (2) (2024) http://dx.doi.org/10.1007/s10710-024-09488-0, URL https://doi.org/10.1007/s10710-024-09488-0.

[3] T. Tohme, D. Liu, K. Youcef-Toumi, GSR: A generalized symbolic regression approach, Trans. Mach. Learn. Res. 2023 (2023) URL https://openreview.net/forum?id=lheUXtDNvP.

[4] Z. Huang, Y. Mei, J. Zhong, Semantic linear genetic programming for symbolic regression, IEEE Trans. Cybern. (2022) 1–14, http://dx.doi.org/10.1109/TCYB.2022.3181461.

[5] Z. Zojaji, M.M. Ebadzadeh, H. Nasiri, Semantic schema based genetic programming for symbolic regression, Appl. Soft Comput. 122 (2022) 108825, http://dx.doi.org/10.1016/j.asoc.2022.108825, URL https://doi.org/10.1016/j.asoc.2022.108825.

[6] C.A. Owen, G. Dick, P.A. Whigham, Characterizing genetic programming error through extended bias and variance decomposition, IEEE Trans. Evol. Comput. 24 (6) (2020) 1164–1176, http://dx.doi.org/10.1109/TEVC.2020.2990626.

[7] L.F. Miranda, V.B.O. Luiz Otavio, B.S.M. Joao Francisco, G.L. Pappa, Instance selection for geometric semantic genetic programming, in: 2020 IEEE Congress on Evolutionary Computation, CEC, 2020, pp. 1–8, http://dx.doi.org/10.1109/CEC48606.2020.9185867.

[8] A. Agapitos, R. Loughran, M. Nicolau, S. Lucas, M. O'Neill, A. Brabazon, A survey of statistical machine learning elements in genetic programming, IEEE Trans. Evol. Comput. 23 (6) (2019) 1029–1048, http://dx.doi.org/10.1109/TEVC.2019.2900916.

[9] J.F.B. Martins, L.O.V. Oliveira, L.F. Miranda, F. Casadei, G.L. Pappa, Solving the exponential growth of symbolic regression trees in geometric semantic genetic programming, in: GECCO 2018 - Proceedings of the 2018 Genetic and Evolutionary Computation Conference, 2018, pp. 1151–1158, http://dx.doi.org/10.1145/3205455.3205593.

[10] L.O.V. Oliveira, F.E. Otero, G.L. Pappa, A dispersion operator for geometric semantic genetic programming, in: GECCO 2016 - Proceedings of the 2016 Genetic and Evolutionary Computation Conference, 2016, pp. 773–780, http://dx.doi.org/10.1145/2908812.2908923.

[11] A. Moraglio, K. Krawiec, C.G. Johnson, Geometric semantic genetic programming, in: International Conference on Parallel Problem Solving from Nature, 2012.

[12] D. Angelis, F. Sofos, T.E. Karakasidis, Artificial intelligence in physical sciences: Symbolic regression trends and perspectives, Arch. Comput. Methods Eng. 30 (6) (2023) 3845–3865, http://dx.doi.org/10.1007/s11831-023-09922-z, URL https://doi.org/10.1007/s11831-023-09922-z.

[13] Y. Bi, B. Xue, M. Zhang, Using a small number of training instances in genetic programming for face image classification, Inform. Sci. 593 (2022) 488–504.

[14] M. Schmidt, H. Lipson, Distilling free-form natural laws from experimental data, Science 324 (5923) (2009) 81–85, http://dx.doi.org/10.1126/science.1165893.

[15] G.E. Hinton, R.R. Salakhutdinov, Reducing the dimensionality of data with neural networks, Science 313 (5786) (2006) 504–507, http://dx.doi.org/10.1126/science.1127647.

[16] H. Majeed, A. Wali, M. Beg, Optimizing genetic programming by exploiting semantic impact of sub trees, Swarm Evol. Comput. 65 (April) (2021) 100923, http://dx.doi.org/10.1016/j.swevo.2021.100923, URL https://doi.org/10.1016/j.swevo.2021.100923.

[17] H. Zhang, A. Zhou, H. Qian, H. Zhang, PS-Tree: A piecewise symbolic regression tree, Swarm Evol. Comput. 71 (January) (2022) 101061, http://dx.doi.org/10.1016/j.swevo.2022.101061, URL https://doi.org/10.1016/j.swevo.2022.101061.

[18] M. Virgolin, T. Alderliesten, C. Witteveen, P.A.N. Bosman, Improving model-based genetic programming for symbolic regression of small expressions, Evol. Comput. 29 (2) (2021) 211–237, http://dx.doi.org/10.1162/evco_a_00278, URL https://doi.org/10.1162/evco_a_00278, arXiv:https://direct.mit.edu/evco/article-pdf/29/2/211/1921067/evco_a_00278.pdf.

[19] M. Virgolin, A. De Lorenzo, F. Randone, E. Medvet, M. Wahde, Model learning with personalized interpretability estimation (ML-PIE), in: GECCO 2021 Companion - Proceedings of the 2021 Genetic and Evolutionary Computation Conference Companion, 2021, pp. 1355–1364, http://dx.doi.org/10.1145/3449726.3463166.

[20] T. Stephens, gplearn: Genetic programming in python, with a scikit-learn inspired API, 2019, Version: gplearn 0.4.1 URL http://gplearn.readthedocs.io/.

[21] F.O. de Franca, G.S.I. Aldeia, Interaction–transformation evolutionary algorithm for symbolic regression, Evol. Comput. 29 (3) (2021) 367–390, http://dx.doi.org/10.1162/evco_a_00285, URL https://doi.org/10.1162/evco_a_00285, arXiv:https://direct.mit.edu/evco/article-pdf/29/3/367/1959462/evco_a_00285.pdf.

[22] M. Virgolin, T. Alderliesten, P.A.N. Bosman, Linear scaling with and within semantic backpropagation-based genetic programming for symbolic regression, in: Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 1084–1092, http://dx.doi.org/10.1145/3321707.3321758, URL https://doi.org/10.1145/3321707.3321758.

[23] I. Arnaldo, K. Krawiec, U.-M. O'Reilly, Multiple regression genetic programming, in: Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation, GECCO '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 879–886, http://dx.doi.org/10.1145/2576768.2598291, URL https://doi.org/10.1145/2576768.2598291.

[24] T. Mundhenk, M. Landajuela, R. Glatt, C.P. Santiago, D. faissol, B.K. Petersen, Symbolic regression via deep reinforcement learning enhanced genetic programming seeding, in: M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, J.W. Vaughan (Eds.), Advances in Neural Information Processing Systems, vol. 34, Curran Associates, Inc., 2021, pp. 24912–24923, URL https://proceedings.neurips.cc/paper_files/paper/2021/file/d073bb8d0c47f317dd39de9c9f004e9d-Paper.pdf.

[25] W. La Cava, P. Orzechowski, B. Burlacu, F. de Franca, M. Virgolin, Y. Jin, M. Kommenda, J. Moore, Contemporary symbolic regression methods and their relative performance, in: J. Vanschoren, S. Yeung (Eds.), Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, vol. 1, Curran, 2021, URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/c0c7c76d30bd3dcaefc96f40275bdc0a-Paper-round1.pdf.

[26] F.O. De França, Transformation-interaction-rational representation for symbolic regression: A detailed analysis of SRBench results, ACM Trans. Evol. Learn. Optim. 3 (2) (2023) http://dx.doi.org/10.1145/3597312.

[27] M. Virgolin, S.P. Pissis, Symbolic Regression is NP-hard, 2016, 2022, pp. 1–9, URL arXiv:2207.01018.

[28] L.F. dal Piccol Sotto, V.V. de Melo, Studying bloat control and maintenance of effective code in linear genetic programming for symbolic regression, Neurocomputing 180 (2016) 79–93, Progress in Intelligent Systems Design, https://doi.org/10.1016/j.neucom.2015.10.109.

[29] S. Silva, S. Dignum, L. Vanneschi, Operator equalisation for bloat free genetic programming and a survey of bloat control methods, Genet. Program. Evol. Mach. 13 (2) (2012) 197–238, http://dx.doi.org/10.1007/s10710-011-9150-5.

[30] M.D. Schmidt, H. Lipson, Co-evolution of fitness maximizers and fitness predictors, Collie. Mech. Electr. Technol. (2010).

[31] S. Silva, E. Costa, Dynamic limits for bloat control in genetic programming and a review of past and current bloat theories, Genet. Program. Evol. Mach. 10 (2) (2009) 141–179, http://dx.doi.org/10.1007/s10710-008-9075-9.

[32] M.D. Schmidt, H. Lipson, Coevolution of fitness predictors, IEEE Trans. Evol. Comput. 12 (6) (2008) 736–749, http://dx.doi.org/10.1109/TEVC.2008.919006.

[33] B. Dolin, F.H.B. Iii, E.G. Rieffel, Co-evolving an effective fitness sample: experiments in symbolic regression and distributed robot control, in: Acm Symposium on Applied Computing, 2002.

[34] J. Eggermont, Bnvki, Stepwise adaptation of weights for symbolic regression with genetic programming, in: Twelveth Belgium/Netherlands Conference on Artificial Intelligence, 2000.

[35] Koza, JohnR, Genetic programming : on the programming of computers by means of natural selection, 1992.

[36] B. Burlacu, K. Yang, M. Affenzeller, Population diversity and inheritance in genetic programming for symbolic regression, in: Natural Computing, vol. 0123456789, Springer Netherlands, 2023, http://dx.doi.org/10.1007/s11047-022-09934-x, URL https://doi.org/10.1007/s11047-022-09934-x.

[37] L. Vanneschi, M. Castelli, S. Silva, Measuring bloat, overfitting and functional complexity in genetic programming, in: Proceedings of the 12th Annual Conference on Genetic and Evolutionary Computation, GECCO '10, ACM, New York, NY, USA, 2010, pp. 877–884, http://dx.doi.org/10.1145/1830483.1830643.

[38] E.J. Vladislavleva, G.F. Smits, D. den Hertog, Order of nonlinearity as a complexity measure for models generated by symbolic regression via Pareto genetic programming, IEEE Trans. Evol. Comput. 13 (2) (2009) 333–349, http://dx.doi.org/10.1109/TEVC.2008.926486.

[39] J. Chen, S. Cai, Y. Wang, W. Xu, J. Ji, M. Yin, Improved local search for the minimum weight dominating set problem in massive graphs by using a deep optimization mechanism, Artificial Intelligence 314 (2023) 103819, http://dx.doi.org/10.1016/j.artint.2022.103819, URL https://doi.org/10.1016/j.artint.2022.103819.

[40] Q. Chen, B. Xue, M. Zhang, Improving generalization of genetic programming for symbolic regression with angle-driven geometric semantic operators, IEEE Trans. Evol. Comput. 23 (3) (2019) 488–502, http://dx.doi.org/10.1109/TEVC.2018.2869621.

[41] M.D. Schmidt, H. Lipson, Solving iterated functions using genetic programming, in: Proceedings of the 11th Annual Conference Companion on Genetic and Evolutionary Computation Conference: Late Breaking Papers, GECCO '09, ACM, New York, NY, USA, 2009, pp. 2149–2154, http://dx.doi.org/10.1145/1570256.1570292.

[42] S.S. Mousavi Astarabadi, M.M. Ebadzadeh, A decomposition method for symbolic regression problems, Appl. Soft Comput. 62 (2018) 514–523, http://dx.doi.org/10.1016/j.asoc.2017.10.041, URL https://doi.org/10.1016/j.asoc.2017.10.041.

[43] W. La Cava, K. Danai, L. Spector, Inference of compact nonlinear dynamic models by epigenetic local search, Eng. Appl. Artif. Intell. 55 (2016) 292–306, http://dx.doi.org/10.1016/j.engappai.2016.07.004, URL https://www.sciencedirect.com/science/article/pii/S0952197616301294.

[44] S.S.M. Astarabadi, M.M. Ebadzadeh, A decomposition method for symbolic regression problems, Appl. Soft Comput. 62 (2018) 514–523, http://dx.doi.org/10.1016/j.asoc.2017.10.041.

[45] J. Wei, Z. Yuan, A generalized benders decomposition-based global optimization approach to symbolic regression for explicit surrogate modeling from limited data information, Comput. Chem. Eng. 142 (2020) 107051, http://dx.doi.org/10.1016/j.compchemeng.2020.107051, URL https://doi.org/10.1016/j.compchemeng.2020.107051.

[46] A.M. Geoffrion, Generalized Benders decomposition, J. Optim. Theory Appl. 10 (4) (1972) 237–260.

[47] C. Gathercole, P. Ross, Dynamic training subset selection for supervised learning in genetic programming, in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 866 LNCS, 1994, pp. 312–321, http://dx.doi.org/10.1007/3-540-58484-6_275.

[48] A. Kattan, A. Agapitos, Y.-S. Ong, A.A. Alghamedi, M. O'Neill, GP made faster with semantic surrogate modelling, Inform. Sci. 355–356 (2016) 169–185, http://dx.doi.org/10.1016/j.ins.2016.03.030.

[49] D. Rivero, E. Fernandez-Blanco, C. Fernandez-Lozano, A. Pazos, Population subset selection for the use of a validation dataset for overfitting control in genetic programming, J. Exp. Theor. Artif. Intell. 32 (2) (2020) 243–271, http://dx.doi.org/10.1080/0952813X.2019.1647562, URL https://doi.org/10.1080/0952813X.2019.1647562.

[50] S.B. Hamida, H. Hmida, A. Borgi, M. Rukoz, Adaptive sampling for active learning with genetic programming, Cogn. Syst. Res. 65 (January 2021) (2020) 23–39.

[51] S.B. Hamida, W. Abdelmalek, F. Abid, Applying dynamic training-subset selection methods using genetic programming for forecasting implied volatility, Comput. Intell. 32 (3) (2016) 369–390, http://dx.doi.org/10.1111/coin.12057, arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/coin.12057, URL https://onlinelibrary.wiley.com/doi/abs/10.1111/coin.12057.

[52] A. Uyumaz, A. Danandeh Mehr, E. Kahya, H. Erdem, Rectangular side weirs discharge coefficient estimation in circular channels using linear genetic programming approach, J. Hydroinform. 16 (6) (2014) 1318–1330, http://dx.doi.org/10.2166/hydro.2014.112, URL https://doi.org/10.2166/hydro.2014.112, arXiv:https://iwaponline.com/jh/article-pdf/16/6/1318/387480/1318.pdf.

[53] A.D. Mehr, E. Kahya, A Pareto-optimal moving average multigene genetic programming model for daily streamflow prediction, J. Hydrol. 549 (2017).

[54] J. Egklitz, P. Poík, Model selection and overfitting in genetic programming: Empirical study, in: Genetic and Evolutionary Computation Conference, 2015.

[55] C. Gagné, Evolutionary computation for supervised learning, in: GECCO 2013 - Proceedings of the 2013 Genetic and Evolutionary Computation Conference Companion, 2013, pp. 827–843, http://dx.doi.org/10.1145/2464576.2480810.

[56] C. Gagné, M. Schoenauer, M. Parizeau, M. Tomassini, Genetic Programming, Validation Sets, and Parsimony Pressure, Springer, 2006.

[57] J. Kim, J. Kim, S. Yoo, GPGPGPU: Evaluation of parallelisation of genetic programming using GPGPU, in: T. Menzies, J. Petke (Eds.), Search Based Software Engineering, Springer International Publishing, Cham, 2017, pp. 137–142.

[58] J. Žegklitz, P. Pošík, Model selection and overfitting in genetic programming: Empirical study, in: GECCO 2015 - Companion Publication of the 2015 Genetic and Evolutionary Computation Conference, (Cic) 2015, pp. 1527–1528, http://dx.doi.org/10.1145/2739482.2764678.

[59] J. Žegklitz, P. Pošík, Model selection and overfitting in genetic programming: Empirical study [extended version], 2015, arXiv:1504.08168, URL https://arxiv.org/abs/1504.08168.

[60] S. Shao, X. Liu, M. Zhou, J. Zhan, X. Liu, Y. Chu, H. Chen, A GPU-based implementation of an enhanced GEP algorithm, in: GECCO'12 - Proceedings of the 14th International Conference on Genetic and Evolutionary Computation, 2012, pp. 999–1006, http://dx.doi.org/10.1145/2330163.2330302.

[61] A.S. Sambo, R.M.A. Azad, Y. Kovalchuk, V.P. Indramohan, H. Shah, Evolving simple and accurate symbolic regression models via asynchronous parallel computing, Appl. Soft Comput. 104 (2021) 107198, http://dx.doi.org/10.1016/j.asoc.2021.107198, URL https://doi.org/10.1016/j.asoc.2021.107198.

[62] G. Nadizar, E. Medvet, D.G. Wilson, Naturally interpretable control policies via graph-based genetic programming, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 14631 LNCS, Springer Nature Switzerland, 2024, pp. 73–89, http://dx.doi.org/10.1007/978-3-031-56957-9_5, URL https://doi.org/10.1007/978-3-031-56957-9_5.

[63] V. Caetano, M.C. Teixeira, G.L. Pappa, Symbolic regression trees as embedded representations, in: GECCO 2023 - Proceedings of the 2023 Genetic and Evolutionary Computation Conference, 2023, pp. 411–419, http://dx.doi.org/10.1145/3583131.3590423.

[64] C. Haider, F.O. de Franca, B. Burlacu, G. Kronberger, Shape-constrained multi-objective genetic programming for symbolic regression, Appl. Soft Comput. 132 (2023) 109855, http://dx.doi.org/10.1016/j.asoc.2022.109855, URL https://doi.org/10.1016/j.asoc.2022.109855.

[65] S. Arslan, N. Kütük, Symbolic regression with feature selection of dye biosorption from an aqueous solution using pumpkin seed husk using evolutionary computation-based automatic programming methods, Expert Syst. Appl. 231 (February) (2023) 120676, http://dx.doi.org/10.1016/j.eswa.2023.120676, URL https://doi.org/10.1016/j.eswa.2023.120676.

[66] D. Rivero, E. Fernandez-Blanco, A. Pazos, DoME: A deterministic technique for equation development and Symbolic Regression, Expert Syst. Appl. 198 (January) (2022) 116712, http://dx.doi.org/10.1016/j.eswa.2022.116712, URL https://doi.org/10.1016/j.eswa.2022.116712.

[67] S. Stijven, W. Minnebo, K. Vladislavleva, Separating the wheat from the chaff: On feature selection and feature importance in regression random forests and symbolic regression, in: Genetic and Evolutionary Computation Conference, GECCO'11 - Companion Publication, 2011, pp. 623–630, http://dx.doi.org/10.1145/2001858.2002059.

[68] Q. Chen, M. Zhang, B. Xue, Feature selection to improve generalization of genetic programming for high-dimensional symbolic regression, IEEE Trans. Evol. Comput. 21 (5) (2017) 792–806, http://dx.doi.org/10.1109/TEVC.2017.2683489.

[69] B. Doerr, A. Lissovoi, P.S. Oliveto, (1+1) genetic programming with functionally complete instruction sets can evolve boolean conjunctions and disjunctions with arbitrarily small error, Artificial Intelligence 319 (2023) 103906, http://dx.doi.org/10.1016/j.artint.2023.103906, URL https://doi.org/10.1016/j.artint.2023.103906.

[70] G.C. Lelis, W.T. Fonseca, A.H. de Lima, A.K. Okazaki, E.C. Figueiredo, A. Riul Jr., G.R. Schleder, P. Samorì, R.F. de Oliveira, Harnessing small-molecule analyte detection in complex media: Combining molecularly imprinted polymers, electrolytic transistors, and machine learning, ACS Appl. Mater. Interf. (2023) http://dx.doi.org/10.1021/acsami.3c16699, Article ASAP.

[71] N.M. Rodrigues, J.E. Batista, W. La Cava, L. Vanneschi, S. Silva, SLUG: Feature selection using genetic algorithms and genetic programming, in: Genetic Programming: 25th European Conference, EuroGP 2022, Held As Part of EvoStar 2022, Madrid, Spain, April 20–22, 2022, Proceedings, Springer-Verlag, Berlin, Heidelberg, 2022, pp. 68–84, http://dx.doi.org/10.1007/978-3-031-02056-8_5, URL https://doi.org/10.1007/978-3-031-02056-8_5.

[72] M. Kommenda, B. Burlacu, G. Kronberger, M. Affenzeller, Parameter identification for symbolic regression using nonlinear least squares, Genet. Program. Evol. Mach. 21 (3) (2020) 471–501, http://dx.doi.org/10.1007/s10710-019-09371-3, URL https://doi.org/10.1007/s10710-019-09371-3.

[73] B. Burlacu, G. Kronberger, M. Kommenda, Operon C++: an efficient genetic programming framework for symbolic regression, in: Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion, GECCO '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 1562–1570, http://dx.doi.org/10.1145/3377929.3398099, URL https://doi.org/10.1145/3377929.3398099.

[74] M. Amane, K. Aissaoui, M. Berrada, Enhancing learning object analysis through fuzzy C-means clustering and web mining methods, Emerg. Sci. J. 7 (3) (2023) 799–807, http://dx.doi.org/10.28991/ESJ-2023-07-03-010.

[75] Anni, Suharjito, Lip-reading with visual form classification using residual networks and bidirectional gated recurrent units, HighTech Innov. J. 4 (2) (2023) 375–386, http://dx.doi.org/10.28991/HIJ-2023-04-02-010.

[76] P. Thanathamathee, S. Sawangarreerak, P. Kongkla, D.N.M. Nizam, An optimized machine learning and deep learning framework for facial and masked facial recognition, Emerg. Sci. J 7 (4) (2023) 1173–1187, http://dx.doi.org/10.28991/ESJ-2023-07-04-010.

[77] J. Weng, Symbolic models and emergent models: A review, IEEE Trans. Auton. Ment. Dev. 4 (1) (2012) 29–53, http://dx.doi.org/10.1109/TAMD.2011.2159113.

[78] S. Dignum, R. Poli, Generalisation of the limiting distribution of program sizes in tree-based genetic programming and analysis of its effects on bloat, in: Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation, GECCO '07, Association for Computing Machinery, New York, NY, USA, 2007, pp. 1588–1595, http://dx.doi.org/10.1145/1276958.1277277, URL https://doi.org/10.1145/1276958.1277277.

[79] S. Luke, L. Panait, A comparison of bloat control methods for genetic programming, Evol. Comput. 14 (3) (2006) 309–344, http://dx.doi.org/10.1162/evco.2006.14.3.309, URL https://doi.org/10.1162/evco.2006.14.3.309, arXiv:https://direct.mit.edu/evco/article-pdf/14/3/309/1493675/evco.2006.14.3.309.pdf.

[80] R. Poli, A simple but theoretically-motivated method to control bloat in genetic programming, in: C. Ryan, T. Soule, M. Keijzer, E. Tsang, R. Poli, E. Costa (Eds.), Genetic Programming, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, pp. 204–217.

[81] R. Crawford-Marks, L. Spector, Size control via size fair genetic operators in the PushGP genetic programming system, in: Proceedings of the 4th Annual Conference on Genetic and Evolutionary Computation, GECCO '02, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2002, pp. 733–739.

[82] A. Ekart, S.Z. Nemeth, Selection Based on the Pareto Nondomination Criterion for Controlling Code Growth in Genetic Programming, Genet. Program. Evol. Mach. 2 (1) (2001) 61–73, URL http://www.springerlink.com/index/j70v0303523v6x65.pdf.

[83] T. Soule, J. Foster, Removal bias: a new cause of code growth in tree based evolutionary programming, in: 1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence (Cat. No.98TH8360), 1998, pp. 781–786, http://dx.doi.org/10.1109/ICEC.1998.700151.

[84] B.-T. Zhang, H. Mühlenbein, Balancing Accuracy and Parsimony in Genetic Programming, Evol. Comput. 3 (1) (1995) 17–38, http://dx.doi.org/10.1162/evco.1995.3.1.17.

[85] N.F. McPhee, J.D. Miller, Accurate replication in genetic programming, in: Proceedings of the 6th International Conference on Genetic Algorithms, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1995, pp. 303–309.

[86] L. Altenberg, The evolution of evolvability in genetic programming, in: Advances in Genetic Programming, MIT Press, Cambridge, MA, USA, 1994, pp. 47–74.

[87] W.A. Tackett, Recombination, Selection, and the Genetic Construction of Computer Programs (Ph.D. thesis), University of Southern California, USA, 1994, Not available from Univ. Microfilms Int.

[88] S. Dignum, R. Poli, Operator equalisation and bloat free GP, in: Proceedings of the 11th European Conference on Genetic Programming, EuroGP '08, Springer-Verlag, Berlin, Heidelberg, 2008, pp. 110–121.

[89] R. Poli, General schema theory for genetic programming with subtree-swapping crossover, in: J. Miller, M. Tomassini, P.L. Lanzi, C. Ryan, A.G.B. Tettamanzi, W.B. Langdon (Eds.), Genetic Programming, Springer Berlin Heidelberg, Berlin, Heidelberg, 2001, pp. 143–159.

[90] R. Poli, Covariant tarpeian method for bloat control in genetic programming, in: R. Riolo, T. McConaghy, E. Vladislavleva (Eds.), Genetic Programming Theory and Practice VIII, Springer New York, New York, NY, 2011, pp. 71–89, http://dx.doi.org/10.1007/978-1-4419-7747-2_5, URL https://doi.org/10.1007/978-1-4419-7747-2_5.

[91] S. Mahler, D. Robilliard, C. Fonlupt, Tarpeian bloat control and generalization accuracy, in: M. Keijzer, A. Tettamanzi, P. Collet, J. van Hemert, M. Tomassini (Eds.), Genetic Programming, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 203–214.

[92] N.F. McPhee, A. Jarvis, E.F. Crane, On the strength of size limits in linear genetic programming, in: K. Deb (Ed.), Genetic and Evolutionary Computation – GECCO 2004, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 593–604.

[93] A.S. Sambo, R.M.A. Azad, Y. Kovalchuk, V.P. Indramohan, H. Shah, Leveraging asynchronous parallel computing to produce simple genetic programming computational models, in: Proceedings of the 35th Annual ACM Symposium on Applied Computing, SAC '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 521–528, http://dx.doi.org/10.1145/3341105.3373921, URL https://doi.org/10.1145/3341105.3373921.

[94] F. Fernández de Vega, G. Olague, D. Lanza, F. Chávez de la O, W. Banzhaf, E. Goodman, J. Menendez-Clavijo, A. Martinez, Time and individual duration in genetic programming, IEEE Access 8 (2020) 38692–38713, http://dx.doi.org/10.1109/ACCESS.2020.2975753.

[95] K. Kinnear, Fitness landscapes and difficulty in genetic programming, in: Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence, vol. 1, 1994, pp. 142–147, http://dx.doi.org/10.1109/ICEC.1994.350026.

[96] P.J. Angeline, An investigation into the sensitivity of genetic programming to the frequency of leaf selection during subtree crossover, in: Proceedings of the 1st Annual Conference on Genetic Programming, MIT Press, Cambridge, MA, USA, 1996, pp. 21–29.

[97] N.Q. Uy, N.X. Hoai, M. O'Neill, R.I. McKay, E. Galván-López, Semantically-based crossover in genetic programming: application to real-valued symbolic regression, Genet. Program. Evol. Mach 12 (2) (2011) 91–119, http://dx.doi.org/10.1007/s10710-010-9121-2.

[98] D. Jackson, Phenotypic diversity in initial genetic programming populations, in: A.I. Esparcia-Alcázar, A. Ekárt, S. Silva, S. Dignum, A.Ş. Uyar (Eds.), Genetic Programming, Springer Berlin Heidelberg, Berlin, Heidelberg, 2010, pp. 98–109.

[99] L. Beadle, C.G. Johnson, Semantic analysis of program initialisation in genetic programming, Genet. Program. Evol. Mach. 10 (3) (2009) 307–337, http://dx.doi.org/10.1007/s10710-009-9082-5.

[100] L. Beadle, C.G. Johnson, Semantically driven mutation in genetic programming, in: 2009 IEEE Congress on Evolutionary Computation, 2009, pp. 1336–1342, http://dx.doi.org/10.1109/CEC.2009.4983099.

[101] K. Krawiec, P. Lichocki, Approximating geometric crossover in semantic space, in: Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation, GECCO '09, Association for Computing Machinery, New York, NY, USA, 2009, pp. 987–994, http://dx.doi.org/10.1145/1569901.1570036, URL https://doi.org/10.1145/1569901.1570036.

[102] K. Krawiec, B. Wieloch, Analysis of semantic modularity for genetic programming, Found. Comput. Decision Sci. (2009) 265–285, URL https://api.semanticscholar.org/CorpusID:8976246.

[103] L. Beadle, C.G. Johnson, Semantically driven crossover in genetic programming, in: 2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence), 2008, pp. 111–116, http://dx.doi.org/10.1109/CEC.2008.4630784.

[104] A. Moraglio, Towards a Geometric Unification of Evolutionary Algorithms (Ph.D. thesis), Department of Computer Science, University of Essex, UK, 2007, URL http://eden.dei.uc.pt/~moraglio/Thesis_final.pdf.

[105] A. Moraglio, R. Poli, Topological interpretation of crossover, in: Annual Conference on Genetic and Evolutionary Computation, 2004, URL https://api.semanticscholar.org/CorpusID:16147928.

[106] Q. Chen, B. Xue, Y. Mei, M. Zhang, Geometric semantic crossover with an angle-aware mating scheme in genetic programming for symbolic regression, in: J. McDermott, M. Castelli, L. Sekanina, E. Haasdijk, P. García-Sánchez (Eds.), Genetic Programming, Springer International Publishing, Cham, 2017, pp. 229–245.

[107] Q. Chen, M. Zhang, B. Xue, New geometric semantic operators in genetic programming: perpendicular crossover and random segment mutation, in: Proceedings of the Genetic and Evolutionary Computation Conference Companion, GECCO '17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 223–224, http://dx.doi.org/10.1145/3067695.3076008, URL https://doi.org/10.1145/3067695.3076008.

[108] N.Q. Uy, N.T. Hien, N.X. Hoai, M. O'Neill, Improving the generalisation ability of genetic programming with semantic similarity based crossover, in: Proceedings of the 13th European Conference on Genetic Programming, EuroGP '10, Springer-Verlag, Berlin, Heidelberg, 2010, pp. 184–195, http://dx.doi.org/10.1007/978-3-642-12148-7_16, URL https://doi.org/10.1007/978-3-642-12148-7_16.

[109] I. Gonçalves, S. Silva, C.M. Fonseca, On the generalization ability of geometric semantic genetic programming, in: European Conference on Genetic Programming, 2015, URL https://api.semanticscholar.org/CorpusID:6878874.

[110] L. Vanneschi, M. Castelli, L. Manzoni, S. Silva, A new implementation of geometric semantic GP and its application to problems in pharmacokinetics, in: Proceedings of the 16th European Conference on Genetic Programming, EuroGP '13, Springer-Verlag, Berlin, Heidelberg, 2013, pp. 205–216, http://dx.doi.org/10.1007/978-3-642-37207-0_18, URL https://doi.org/10.1007/978-3-642-37207-0_18.

[111] F.O. de França, Alleviating overfitting in transformation-interaction-rational symbolic regression with multi-objective optimization, Genet. Program. Evol. Mach. 24 (2) (2023) 1–23, http://dx.doi.org/10.1007/s10710-023-09461-3, URL https://doi.org/10.1007/s10710-023-09461-3.

[112] Q. Chen, B. Xue, M. Zhang, Preserving population diversity based on transformed semantics in genetic programming for symbolic regression, IEEE Trans. Evol. Comput. 25 (3) (2021) 433–447, http://dx.doi.org/10.1109/TEVC.2020.3046569.

[113] M. Nicolau, A. Agapitos, Choosing function sets with better generalisation performance for symbolic regression models, Genet. Program. Evol. Mach. 22 (1) (2021) 73–100, http://dx.doi.org/10.1007/s10710-020-09391-4, URL https://doi.org/10.1007/s10710-020-09391-4.

[114] S. Silva, V. Ingalalli, S. Vinga, J.M. Carreiras, J.B. Melo, M. Castelli, L. Vanneschi, I. Gonçalves, J. Caldas, Prediction of forest aboveground biomass: An exercise on avoiding overfitting, in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), 7835 LNCS, 2013, pp. 407–417, http://dx.doi.org/10.1007/978-3-642-37192-9_41.

[115] I. Gonçalves, S. Silva, J.B. Melo, J.M. Carreiras, Random sampling technique for overfitting control in genetic programming, in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 7244 LNCS, 2012, pp. 218–229, http://dx.doi.org/10.1007/978-3-642-29139-5_19.

[116] A. Agapitos, A. Brabazon, M. O'Neill, Controlling overfitting in symbolic regression based on a bias/variance error decomposition, in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 7491 LNCS, (PART 1) 2012, pp. 438–447, http://dx.doi.org/10.1007/978-3-642-32937-1_44.

[117] S.S. Mousavi Astarabadi, M.M. Ebadzadeh, Avoiding overfitting in symbolic regression using the first order derivative of GP trees, in: Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation, in: GECCO Companion '15, ACM, New York, NY, USA, 2015, pp. 1441–1442, http://dx.doi.org/10.1145/2739482.2764662.

[118] M. Amir Haeri, M.M. Ebadzadeh, G. Folino, Improving GP generalization: a variance-based layered learning approach, Genet. Program. Evol. Mach. 16 (1) (2015) 27–55.

[119] J. Ni, P. Rockett, Tikhonov regularization as a complexity measure in multiobjective genetic programming, IEEE Trans. Evol. Comput. 19 (2) (2015) 157–166, http://dx.doi.org/10.1109/TEVC.2014.2306994.

[120] N. Nikolaev, H. Iba, Regularization approach to inductive genetic programming, IEEE Trans. Evol. Comput. 5 (4) (2001) 359–375, http://dx.doi.org/10.1109/4235.942530.

[121] I. Goncalves, S. Silva, Balancing learning and overfitting in genetic programming, Lecture Notes in Comput. Sci. 7831 (2013) 73–84.

[122] L. Vanneschi, S. Silva, Using operator equalisation for prediction of drug toxicity with genetic programming, in: L.S. Lopes, N. Lau, P. Mariano, L.M. Rocha (Eds.), Progress in Artificial Intelligence, Springer Berlin Heidelberg, Berlin, Heidelberg, 2009, pp. 65–76.

[123] M. Castelli, S. Silva, L. Vanneschi, A C++ framework for geometric semantic genetic programming, Genet. Program. Evol. Mach. 16 (1) (2015) 73–81, http://dx.doi.org/10.1007/s10710-014-9218-0.

[124] L. Vanneschi, S. Silva, M. Castelli, L. Manzoni, Geometric semantic genetic programming for real life applications, in: Genetic Programming Theory and Practice, 2013, URL https://api.semanticscholar.org/CorpusID:40595000.

[125] Y. Liu, T. Khoshgoftaar, Reducing overfitting in genetic programming models for software quality classification, in: Eighth IEEE International Symposium on High Assurance Systems Engineering, 2004.

[126] W.B. Langdon, Minimising testing in genetic programming department of computer science minimising testing in genetic programming, Search (April) (2011).

[127] I. Goncalves, S. Silva, Experiments on controlling overfitting in genetic programming, in: EPIA2011 - 15th Portuguese Conference on Artificial Intelligence, (October 2011) 2011, pp. 978–989.

[128] D. Robilliard, C. Fonlupt, Backwarding: An overfitting control for genetic programming in a remote sensing application, in: P. Collet, C. Fonlupt, J.-K. Hao, E. Lutton, M. Schoenauer (Eds.), Artificial Evolution, Springer Berlin Heidelberg, Berlin, Heidelberg, 2002, pp. 245–254.

[129] Z. Zhang, C. Dong, On the Effectiveness of Kinoform Operations in Holography, in: ACM International Conference Proceeding Series, 2, 2020, pp. 107–111, http://dx.doi.org/10.1145/3403746.3403913.

[130] C. Mattiussi, M. Waibel, D. Floreano, Measures of diversity for populations and distances between individuals with highly reorganizable genomes, Evol. Comput. 12 (4) (2004) 495–515, http://dx.doi.org/10.1162/1063656043138923.

[131] E. Burke, S. Gustafson, G. Kendall, Diversity in genetic programming: an analysis of measures and correlation with fitness, IEEE Trans. Evol. Comput. 8 (1) (2004) 47–62, http://dx.doi.org/10.1109/TEVC.2003.819263.

[132] A.R. Burks, W.F. Punch, An investigation of hybrid structural and behavioral diversity methods in genetic programming, in: R. Riolo, B. Worzel, B. Goldman, B. Tozier (Eds.), Genetic Programming Theory and Practice XIV, Springer International Publishing, Cham, 2018, pp. 19–34, http://dx.doi.org/10.1007/978-3-319-97088-2_2, URL https://doi.org/10.1007/978-3-319-97088-2_2.

[133] N.F. McPhee, N.J. Hopper, Analysis of genetic diversity through population history, in: Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation - Volume 2, GECCO '99, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1999, pp. 1112–1120.

[134] S. Khanchi, M.I. Heywood, N. Zincir-Heywood, On the impact of class imbalance in GP streaming classification with label budgets, in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 9594, 2016, pp. 35–50, http://dx.doi.org/10.1007/978-3-319-30668-1_3.

[135] A. Ekárt, S.Z. Németh, Maintaining the diversity of genetic programs, in: Proceedings of the 5th European Conference on Genetic Programming, EuroGP '02, Springer-Verlag, Berlin, Heidelberg, 2002, pp. 162–171.

[136] E. Galván-López, B. Cody-Kenny, L. Trujillo, A. Kattan, Using semantics in the selection mechanism in genetic programming: A simple method for promoting semantic diversity, in: 2013 IEEE Congress on Evolutionary Computation, 2013, pp. 2972–2979, http://dx.doi.org/10.1109/CEC.2013.6557931.

[137] D. Jackson, Promoting phenotypic diversity in genetic programming, in: Proceedings of the 11th International Conference on Parallel Problem Solving from Nature: Part II, PPSN '10, Springer-Verlag, Berlin, Heidelberg, 2010, pp. 472–481.

[138] N.F. McPhee, B. Ohs, T. Hutchison, Semantic building blocks in genetic programming, in: M. O'Neill, L. Vanneschi, S. Gustafson, A.I. Esparcia Alcázar, I. De Falco, A. Della Cioppa, E. Tarantino (Eds.), Genetic Programming, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 134–145.

[139] T.P. Pawlak, K. Krawiec, Semantic geometric initialization, in: European Conference on Genetic Programming, 2016, URL https://api.semanticscholar.org/CorpusID:30517964.

[140] J.D. Knowles, R.A. Watson, On the utility of redundant encodings in mutation-based evolutionary search, in: Proceedings of the 7th International Conference on Parallel Problem Solving from Nature, in: PPSN VII, Springer-Verlag, Berlin, Heidelberg, 2002, pp. 88–98.

[141] G. Murphy, C. Ryan, A simple powerful constraint for genetic programming, in: Proceedings of the 11th European Conference on Genetic Programming, EuroGP '08, Springer-Verlag, Berlin, Heidelberg, 2008, pp. 146–157.

[142] E.O. Scott, K.A. De Jong, Evaluation-time bias in asynchronous evolutionary algorithms, in: Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation, in: GECCO Companion '15, Association for Computing Machinery, New York, NY, USA, 2015, pp. 1209–1212, http://dx.doi.org/10.1145/2739482.2768482, URL https://doi.org/10.1145/2739482.2768482.

[143] D.A. Augusto, H.J. Barbosa, Accelerated parallel genetic programming tree evaluation with OpenCL, J. Parallel Distrib. Comput. 73 (1) (2013) 86–100, http://dx.doi.org/10.1016/j.jpdc.2012.01.012, URL https://doi.org/10.1016/j.jpdc.2012.01.012.

[144] S. Harding, W. Banzhaf, Fast genetic programming on GPUs, in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 4445 LNCS, 2007, pp. 90–101, http://dx.doi.org/10.1007/978-3-540-71605-1_9.

[145] D. Sudholt, Parallel evolutionary algorithms, in: J. Kacprzyk, W. Pedrycz (Eds.), Springer Handbook of Computational Intelligence, Springer Berlin Heidelberg, Berlin, Heidelberg, 2015, pp. 929–959, http://dx.doi.org/10.1007/978-3-662-43505-2_46, URL https://doi.org/10.1007/978-3-662-43505-2_46.

[146] M. Schmidt, H. Lipson, Age-fitness Pareto optimization, in: R. Riolo, T. McConaghy, E. Vladislavleva (Eds.), Genetic Programming Theory and Practice VIII, Springer New York, New York, NY, 2011, pp. 129–146, http://dx.doi.org/10.1007/978-1-4419-7747-2_8, URL https://doi.org/10.1007/978-1-4419-7747-2_8.

[147] M. Oussaidène, B. Chopard, O.V. Pictet, M. Tomassini, Parallel genetic programming and its application to trading model induction, Parallel Comput. 23 (8) (1997) 1183–1198, http://dx.doi.org/10.1016/S0167-8191(97)00045-8, URL https://www.sciencedirect.com/science/article/pii/S0167819197000458.

[148] J.R. Koza, D. Andre, Parallel Genetic Programming on a Network of Transputers Department of Computer Science Transputers, Science (January) (1995).

[149] D. Power, R. C., R. Azad, Promoting diversity using migration strategies in distributed genetic algorithms, in: 2005 IEEE Congress on Evolutionary Computation, vol. 2, 2005, pp. 1831–1838, http://dx.doi.org/10.1109/CEC.2005.1554910.

[150] Z. Huang, J. Zhong, L. Feng, Y. Mei, W. Cai, A fast parallel genetic programming framework with adaptively weighted primitives for symbolic regression, Soft Comput. 24 (10) (2020) 7523–7539, http://dx.doi.org/10.1007/s00500-019-04379-4, URL https://doi.org/10.1007/s00500-019-04379-4.

[151] S.M. Udrescu, A. Tan, J. Feng, O. Neto, T. Wu, M. Tegmark, AI Feynman 2.0: Pareto-optimal symbolic regression exploiting graph modularity, Adv. Neural Inf. Process. Syst. 2020-December (NeurIPS) (2020) 1–17, arXiv:2006.10782.

[152] Y. Jin, W. Fu, J. Kang, J. Guo, J. Guo, Bayesian symbolic regression, 2020, arXiv:1910.08892.

[153] B.K. Petersen, M. Landajuela, T.N. Mundhenk, C.P. Santiago, S.K. Kim, J.T. Kim, Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients, 2021, arXiv:1912.04871.

[154] W. La Cava, T. Helmuth, L. Spector, J.H. Moore, A probabilistic and multi-objective analysis of lexicase selection and ϵ-lexicase selection, Evol. Comput. 27 (3) (2019) 377–402, http://dx.doi.org/10.1162/evco_a_00224, URL https://doi.org/10.1162/evco_a_00224, arXiv:https://direct.mit.edu/evco/article-pdf/27/3/377/1858632/evco_a_00224.pdf.

[155] W.L. Cava, T.R. Singh, J.P. Taggart, S. Suri, J.H. Moore, Learning concise representations for regression by evolving networks of trees, in: 7th International Conference on Learning Representations, ICLR 2019, 2019, pp. 1–16, arXiv:1807.00981.

[156] T. McConaghy, FFX: Fast, scalable, deterministic symbolic regression technology, in: R. Riolo, E. Vladislavleva, J.H. Moore (Eds.), Genetic Programming Theory and Practice IX, Springer New York, New York, NY, 2011, pp. 235–260, http://dx.doi.org/10.1007/978-1-4614-1770-5_13, URL https://doi.org/10.1007/978-1-4614-1770-5_13.

[157] T. Chen, C. Guestrin, Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16, Association for Computing Machinery, New York, NY, USA, 2016, pp. 785–794, http://dx.doi.org/10.1145/2939672.2939785, URL https://doi.org/10.1145/2939672.2939785.

[158] G. Ke, Q. Meng, T. Finely, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems 30, NIP 2017, 2017, URL https://www.microsoft.com/en-us/research/publication/lightgbm-a-highly-efficient-gradient-boosting-decision-tree/.

[159] L. Breiman, Random forests, Mach. Learn. 45 (2001) 5–32, URL https://api.semanticscholar.org/CorpusID:89141.

[160] R.E. Schapire, The boosting approach to machine learning: An overview, in: D.D. Denison, M.H. Hansen, C.C. Holmes, B. Mallick, B. Yu (Eds.), Nonlinear Estimation and Classification, Springer New York, New York, NY, 2003, pp. 149–171, http://dx.doi.org/10.1007/978-0-387-21579-2_9, URL https://doi.org/10.1007/978-0-387-21579-2_9.

[161] M. Kommenda, G. Kronberger, M. Affenzeller, S.M. Winkler, B. Burlacu, Evolving simple symbolic regression models by multi-objective genetic programming, in: R. Riolo, W. Worzel, M. Kotanchek, A. Kordon (Eds.), Genetic Programming Theory and Practice XIII, Springer International Publishing, Cham, 2016, pp. 1–19, http://dx.doi.org/10.1007/978-3-319-34223-8_1, URL https://doi.org/10.1007/978-3-319-34223-8_1.

[162] M. Virgolin, A. De Lorenzo, E. Medvet, F. Randone, Learning a formula of interpretability to learn interpretable formulas, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 12270 LNCS, Springer International Publishing, 2020, pp. 79–93, http://dx.doi.org/10.1007/978-3-030-58115-2_6, URL https://doi.org/10.1007/978-3-030-58115-2_6, arXiv:2004.11170.

[163] W.J. Murdoch, C. Singh, K. Kumbier, R. Abbasi-Asl, B. Yu, Definitions, methods, and applications in interpretable machine learning, Proc. Natl. Acad. Sci. USA 116 (44) (2019) 22071–22080, http://dx.doi.org/10.1073/pnas.1900654116.

[164] A. Kattan, Y.-S. Ong, Surrogate genetic programming: A semantic aware evolutionary search, Inform. Sci. 296 (2015) 345–359, http://dx.doi.org/10.1016/j.ins.2014.10.053.

[165] Y.Z. Peng, C.A. Yuan, X. Qin, J.T. Huang, Y.B. Shi, An improved Gene Expression Programming approach for symbolic regression problems, Neurocomputing 137 (2014) 293–301, http://dx.doi.org/10.1016/j.neucom.2013.05.062, URL https://doi.org/10.1016/j.neucom.2013.05.062.

[166] P. Machado, M.I. Heywood, J. McDermott, M. Castelli, P. García-Sánchez, P. Burelli, S. Risi, K. Sim, Genetic programming 18th European Conference, EuroGP 2015 Copenhagen, Denmark, April 8–10, 2015 Proceedings, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 9025, 2015.

[167] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Trans. Evol. Comput. 6 (2) (2002) 182–197, http://dx.doi.org/10.1109/4235.996017.

[168] E.B. De Lima, G.L. Pappa, J.M. De Almeida, M.A. Goncalves, W. Meira, Tuning genetic programming parameters with factorial designs, in: 2010 IEEE World Congress on Computational Intelligence, WCCI 2010 - 2010 IEEE Congress on Evolutionary Computation, CEC 2010, 2010, http://dx.doi.org/10.1109/CEC.2010.5586084.

[169] A. Piszcz, T. Soule, Genetic programming: Parametric analysis of structure altering mutation techniques, Stud. Comput. Intell. 54 (2007) 143–160, http://dx.doi.org/10.1007/978-3-540-69432-8_7.

[170] J.A. Walker, J.F. Miller, The automatic acquisition, evolution and reuse of modules in Cartesian genetic programming, IEEE Trans. Evol. Comput. 12 (4) (2008) 397–417, http://dx.doi.org/10.1109/TEVC.2007.903549.

[171] M. Keijzer, Improving symbolic regression with interval arithmetic and linear scaling, in: C. Ryan, T. Soule, M. Keijzer, E. Tsang, R. Poli, E. Costa (Eds.), Genetic Programming, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, pp. 70–82.

[172] M. Korns, Accuracy in symbolic regression, ISBN: 978-1-4614-1769-9, 2011, pp. 129–151, http://dx.doi.org/10.1007/978-1-4614-1770-5_8.
[173] D. Dua, C. Graff, UCI Machine Learning Repository, University of California, Irvine, School of Information and Computer Sciences, 2017, URL http://archive.ics.uci.edu/ml.
[174] J.H. Friedman, Greedy function approximation: A gradient boosting machine, Ann. Statist. 29 (5) (2001) 1189–1232, http://dx.doi.org/10.1214/aos/1013203451.
[175] I. Arnaldo, U.-M. O'Reilly, K. Veeramachaneni, Building predictive models via feature synthesis, in: Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation, GECCO '15, ACM, New York, NY, USA, 2015, pp. 983–990, http://dx.doi.org/10.1145/2739480.2754693.
[176] S.H. Strogatz, Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering, second ed., Westview Press, 2014.